

NCHU - [DRL][FR] Final Proposal Phase 5 Agent 模型交接

media.mp4

檔案

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Smart_City_Drone_Law_Enforcement.pdf

PDF

Frank_oral_defense_0725_v7.pptx

簡報

我想要開發一個 Deep Reinforcement Learning 專案

如附圖 1 ilearning 平台上的描述

但我現在還沒找到要做的主題 請你建議我

請你在完整理解過教授要的 proposal 架構後再推薦我最適合用教授這套 A-to-Z 的論文寫作方式做 **final project proposal**

附檔是教授提供的 reference 請幫我完整全部檔案都看完, 理解後

推薦我最適合套用進此流程的 DRL 題目

附上教授 A-to-Z 的論文寫作流程影片介紹:

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

顯示更多顯示較少

我看完了你貼的 ilearning 作業說明, 也把三個附檔都過了一遍。我的判斷很明確:

最適合你拿來做這門 DRL final project proposal 的題目, 是:

「用 Deep Reinforcement Learning 做都市路口自適應交通號誌控制 (Adaptive Traffic Signal Control)」。

你附的兩份 drone reference, 其實是在示範同一種教授喜歡的敘事方式: 先把問題的重要性講大, 再整理 related work, 接著把方法流程、資料/系統設計、實驗比較、限制與未來導入講清楚。口試簡報版明確就是 **Motivation → Related Works → Proposed Method → Experimental Results → Conclusion**, 還把資料量、模型、限制都量化寫出來; Smart City PDF 則把它壓成「問題—系統流程—實驗—風險—落地路線圖」的簡報故事。至於 2330 那份雖然不是 DRL 論文, 但它很像一個「固定頁面模板 + 關鍵結論 + 證據框 + 決策邏輯」的視覺報告範本, 這對你做 proposal 投影片其實很有參考價值。

Frank_oral_defense_0725_v7 Smart_City_Drone_Law_Enforcement

2330_SMC_FVG_MTF_Report_A4_Port...

再加上你附的影片頁面就是 **Project Proposal A to Z (字幕外加)**; 搭配你上傳的 mp4 內容, 我理解教授現在要的不是「先把系統全部做完」, 而是要你先把整條論文故事線講順: **這個問題為什麼值得做、現在卡在哪裡、你打算怎麼設計、未來要怎麼驗證**。影片裡對 abstract 的要求其實很清楚, 就是 **Attention Getter / Motivation → But → Cure → Development → Experiments → Findings**; 而實驗段落也要先想好 **baseline、metrics、setup、tuning、useful results**。  YouTube

為什麼我最推薦「DRL 自適應交通號誌控制」

第一，它最貼近教授 **reference** 的味道。你手上的 reference 是智慧城市、無人機、執法、系統導入，核心共同點不是「影像辨識」而已，而是**真實城市問題 + AI/系統化設計 + 可驗證的實驗**。交通號誌控制同樣是很典型的智慧城市議題，問題好講、價值大、實驗也能量化。

第二，它非常適合 **A-to-Z** 的 **proposal** 寫法。

這題天然就能寫成：

- Attention Getter / Motivation：都市壅塞、固定時制浪費通行效率
- But：固定式或傳統 rule-based 控制難以適應動態車流
- Cure：用 DRL 根據即時狀態決定號誌切換
- Development：定義 state / action / reward，建立 SUMO 模擬環境
- Experiments：和 fixed-time、actuated control、DQN/PPO 比較
- Findings：看平均等待時間、queue length、travel time 是否下降

第三，它**工具鏈成熟、可重現、風險低**。SUMO 是官方維護的開源微觀交通模擬器，能處理大型路網；SUMO-RL 已經把 Traffic Signal Control 包成可直接拿來做 RL 的環境，而且支援 Gymnasium、Stable-Baselines3、RLlib，也能改 state 和 reward；Gymnasium 本身提供標準 RL API 與自訂環境教學；Stable-Baselines3 則提供穩定的 PyTorch 演算法實作，像 PPO 與 DQN 都可直接用。🌱 [Eclipse SUMO+7](#)

第四，它**很容易控制專案範圍**。你可以先做「單一路口」拿到能跑的結果，再決定要不要擴成「多路口」或「multi-agent」。SUMO-RL 也已經明講支援 multi-agent；PettingZoo 則是 Farama 官方的多智能體 RL 介面標準。也就是說，這題從保守版到進階版都有路可以走。🤖 [Lucas Alegre+1](#)

我會怎麼幫你定這題的正式題目

我建議你不要只寫成很泛的「Traffic Signal Control using DRL」，而是寫成：

中文題目

以需求感知深度強化學習進行都市路口自適應交通號誌控制

英文題目

Demand-Aware Deep Reinforcement Learning for Adaptive Traffic Signal Control at Urban Intersections

這樣的好處是你 proposal 裡的「novelty」比較容易站住腳：

你不是只說「我用 DRL」，而是說你要做 **demand-aware state design** 或 **reward shaping**，讓 agent 不只顧單一方向，而是同時兼顧等待時間、排隊長度與通行效率。

這題最適合的 proposal 骨架

1. Introduction / A-to-F

先講城市壅塞與固定號誌的侷限，再指出現有方法在動態需求、尖離峰切換、不同流向公平性上的不足，最後收斂到你的研究問題：

「能不能讓 DRL 根據即時車流自適應調整號誌，相比固定時制顯著降低等待成本？」

2. Related Work / NotebookLM

文獻建議分三條線整理：

- 傳統交通號誌控制：fixed-time、actuated、rule-based

- RL / DRL traffic signal control : Q-learning、DQN、PPO、multi-agent
- reward / state design : queue、delay、pressure、fairness

教授 reference 不是單純列 paper，而是有「從舊到新、從簡單到複雜」的演進感，這點你要學。

Frank_oral_defense_0725_v7

3. Proposed Design / 規格書

這一段你要寫得像系統設計，而不是只有概念：

- Environment : SUMO / SUMO-RL
- Agent : 先用 DQN 或 PPO
- State : 各車道 queue length、waiting time、current phase、phase elapsed time
- Action : 切換到哪個 phase，或維持 / 切換
- Reward : 負的總等待時間 + queue penalty
- Baselines : fixed-time、actuated control
- Metrics : average waiting time、queue length、throughput、travel time

我建議你的最小可行版本

不要一開始就做太大。最穩的版本是：

- 場景：單一路口
- 演算法：DQN、PPO 二選一，外加 fixed-time baseline
- 目標：讓 average waiting time 明顯下降
- 可選加分點：再加入第二個 reward term，處理 fairness 或尖峰流量

這樣 proposal 好寫，final 也比較做得完。

我不推薦你現在優先選的題目

1. 無人機巡航路徑規劃 DRL

題目很帥，也跟 reference 很像，但實作風險高。AirSim 官方雖然有 RL 範例，但文件自己也說這一塊仍在 active development；你很容易把時間花在模擬器與飛行控制，而不是論文主體。  Microsoft GitHub

2. CARLA 自動駕駛 DRL

CARLA 本身很強，是為自駕研究打造的開源模擬平台；但官方 RL repo 也明寫了只放 inference code、training code 沒有釋出。這種題目工程量通常比課程 final project 大很多。  CARLA Simulator+1

三個候選題目的結論比較

題目	和教授 reference 的契合度	Proposal 好寫程度	實作風險	我建議
DRL 自適應交通號誌控制	9.5/10	9.5/10	4/10	最推薦
DRL 無人機巡航 / 巡檢路徑規劃	8.5/10	7/10	7.5/10	第二順位
DRL 自動駕駛 (CARLA)	7/10	6/10	9/10	不建議當首選

我的最終建議

你的情況下，**最穩、最像教授會買單、最能套進 A-to-Z 流程的，就是「DRL 自適應交通號誌控制」。**

因為它同時滿足四件事：

- 有**真實世界動機**
- 有**明確 baseline 與 metrics**
- 有**成熟工具鏈**
- 有**好寫的論文故事**

你決定選這題的話，我下一則就直接幫你寫成**教授格式的 final project proposal 初稿**，包含：
題目、abstract、introduction、related work 架構、proposed design、experiment plan、預期貢獻。

<https://www.youtube.com/shorts/qeNim1T082Y>

這個盲人眼鏡開源專案 可以嗎
也幫我把 github 位置找出來

可以，但要分開看：

如果你是問「這個題目能不能當 final project proposal」：可以。

如果你是問「可不可以直接當 DRL 專案原封不動拿去做」：我不建議。

你貼的這支 Shorts 看起來對應的是近來流傳的那個「95 後手搓 AI 導盲眼鏡」開源專案；相關文章直接指向同一個 GitHub 倉庫，而該 repo 自己也寫明這是一個面向視障人士的智能導航與輔助系統。  [YouTube+2](#)

這個專案本身其實**很完整，也很有 proposal 故事性**：它整合了盲道導航、過馬路輔助、物品識別查找、即時語音互動；技術上用到 YOLO 分割做盲道辨識、YOLO-E + ByteTrack 做目標追蹤、MediaPipe 做手部引導，並結合 Qwen-Omni 與阿里雲 DashScope 做語音與多模態互動。repo 也標示為 MIT license。  [GitHub+1](#)

但如果你的課是 **Deep Reinforcement Learning**，那這個 repo **目前不是一個 DRL 專案**。我查它的公開 README，找不到 reinforcement learning、RL、DQN、PPO 之類的设计痕跡；從結構看起來，它更像是**電腦視覺 + 多模態語音 + 邊緣裝置整合**的 assistive AI 系統，而不是用 agent 學 policy 的 DRL 研究。  [GitHub+3](#)

所以我的判斷是：

結論一：可以當靈感來源，但不要直接把它當成「DRL 題目」交上去。

因為教授要的是可以用 A-to-Z 論文方式講清楚的 proposal，而這個專案原型雖然很吸睛，卻比較偏系統整合。若你直接照搬，老師很可能會問：「你的 reinforcement learning 在哪裡？」  [GitHub+1](#)

結論二：最好的做法，是把它改寫成一個“以這個導盲眼鏡為應用場景”的 DRL 題目。

例如你可以改成：

- **DRL-based Assistive Navigation Policy for Blind-Path Following and Obstacle Avoidance**
- **Using Deep Reinforcement Learning to Optimize Audio Guidance for Smart Glasses Navigation**
- **A DRL Decision Module for Crosswalk Assistance in Smart Glasses for the Visually Impaired**

這樣就能把原本的 CV 感知模組保留成「環境輸入」，而把真正的研究貢獻放在 **agent 如何根據狀態決定下一步提示/方向/避障策略**。這樣比較符合 DRL 課程定位，也更容易寫成教授喜歡的 A-to-Z 結構。

GitHub 位置

倉庫是：[AI-FanGe/OpenAIGlasses_for_Navigation](#)。文章與 repo 互相對得上，repo 首頁也明確寫它是「a open framework for blind navigation based on esp32」。

另外提醒你兩個現實面問題。第一，repo 自己明寫**僅供交流學習，不要直接給視障人群使用**，所以你 proposal 最好定位成研究原型，不要寫成 ready-to-deploy 產品。第二，它的 README 顯示需要 GPU、CUDA、ESP32-CAM、DashScope API key，而且 quick start 裡的 clone 指令還留著像 placeholder 的路徑，代表專案雖然開源，但工程細節還有不少要自己補。

我的建議會是：

這題可以做，但要改成「導盲眼鏡中的 DRL 決策/導航策略」而不是「開源導盲眼鏡系統重現」。

你若要，我下一步可以直接幫你把这个題目改寫成一份符合教授 A-to-Z 架構的 proposal 題目 + Abstract + Introduction + Related Work + Proposed Design。

貼上的 Markdown (1).md

檔案

好 幫我詳細推薦跟 "網路吞吐量設計應用 DRL" 有關的 能獲得 100 分滿分的期末報告題目
請推薦我 10 個題目並作比較分析
最後推薦你心目中的第一名給我

搜尋前請先看我先前給你的 Final Project Proposal 要求 如附圖所示 我要能適合先提案再收斂成期末專題
且能更做出來的題目

media.mp4

檔案

Smart_City_Drone_Law_Enforcement.pdf

PDF

2330_SMC_FVG_MTF_Report_A4_Portrait.pdf

PDF

Frank_oral_defense_0725_v7.pdf

PDF

From 0 to 1 Deep RL with SMC MDP.pdf

PDF

<https://www.youtube.com/watch?v=MqwmP4CJnSE>

附上三種參考資料(請自行閱讀附檔後自動對應以下範圍) 幫我做成同樣的研究流程

1. A to Z 論文撰寫框架: 有 2 份

2. 專題提案成果: 有 3 份
3. 教授的 chatgpt 收斂出專題提案的對話過程: 有 1 份

我的要求目前只有以下三點 其餘需求請你幫我補充成可以獲得 100 分滿分期末專題提案的程度交付物 並幫我規劃推進步驟:

1. 讓 "以深度強化學習進行單一瓶颈鏈路壅塞控制與吞吐量最佳化" 這題目能產出如 "專題提案成果" 的最終交付物給我
 2. 我的所有程式碼工作都會交給 antigravity 執行工作且要用到工具鏈 **"openspec"**
 3. final project 都會交給 ai agent antigravity 執行 我只負責規劃及下指令 部參與實際程式開發 我開發的是整個專案 這是我的角色
- 顯示更多顯示較少

收到。之後只要你跟我要「prompt」，但沒有特別指定品質，我都會預設用這個等級：**最高品質、可直接餵回執行、任務邊界完整、驗收條件明確、能接到後續階段。**

這份是 **Phase 2 / OpenSpec change 01 : project-charter** 的最佳 prompt。它的設計依據是：OpenSpec 的核心是先把規格變成 single source of truth，再進入實作；官方與相關文件也強調以 proposal / spec / design / tasks 這類 artifacts 讓人與 AI coding assistant 在寫 code 前先對齊。 [GitHub+2](#)

同時，這份 prompt 會把本專題的技術邊界鎖定在 ns-3 / ns3-gym / DRL congestion control 的 Phase 0、Phase 1 定案上；ns-3 官方支援多種 TCP congestion control baseline，ns3-gym 則是為 ns-3 與 RL 整合而設計，這些都會被要求寫進 charter 作為後續 source of truth。 [NSNAM+2](#)

請下一回合直接貼上以下整段：

你現在要執行的是我 DRL Final Project 的 ****Phase 2：用 OpenSpec 建立專案真相來源**** 的第一階段：

OpenSpec change 01:project-charter

最高任務目標

你要基於前面已完成的 Phase 0 與 Phase 1 定案內容，替我產出一份可以作為整個專案 ****source of truth**** 的 OpenSpec-style project charter。

這不是 proposal 草稿，也不是一般摘要，而是後續所有 change 的上游規格母文件。
它必須能讓我、Antigravity、OpenSpec、後續 PPT / GitHub / 10 分鐘影片全部對齊同一個方向。

本階段只產出文件，不寫任何程式碼，不做實作，不啟動 ns-3，不啟動 ns3-gym，不訓練模型。

一、Phase 0 / Phase 1 已凍結內容

請你完整承接以下已定案內容，不得自行改題、擴題或改回其他題目。

1. 正式題目

中文題目

以深度強化學習進行單一瓶颈鏈路壅塞控制與吞吐量最佳化

英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

簡報首頁版題目

DRL-Based Congestion Control over a Bottleneck Link

建議 GitHub repo name
drl-bottleneck-congestion-control

2. 研究定位

本專題是：

- networking systems 題
- DRL sequential decision-making 題
- congestion control / throughput optimization 題
- ns-3 / ns3-gym / Stable-Baselines3 專題
- baseline comparison 專題
- proposal-first / spec-first 專題

本專題不是：

- IPFS 實作題
- QUIC 實作題
- Linux kernel TCP 修改題
- 真實 Internet deployment 題
- multi-agent 題
- multi-path routing 題
- 大型網路拓模題
- Pantheon full integration 題
- 自適應交通號誌控制題

3. 核心研究主張

本研究將單一瓶頸鏈路的壅塞控制建模為深度強化學習問題，讓 agent 根據網路狀態觀測值調整傳輸控制行為，並透過 throughput、RTT、packet loss 與 utility 指標，和 NewReno、CUBIC、BBR 等 TCP baseline 進行比較。

本研究不承諾 DRL 必然全面超越所有 TCP baseline。

成功標準是建立清楚的 DRL formulation、可重現的 baseline / experiment plan、以及能支撐後續實作的 OpenSpec source of truth。

4. 已定案工具鏈

後續 implementation 預設工具鏈為：

- ns-3: network simulator
- ns3-gym: ns-3 與 RL agent 的介面
- Stable-Baselines3: DRL 訓練框架
- DQN: MVP 第一版演算法，因為 action 初始設計為 discrete
- PP0: future extension, 不是 MVP 必做
- OpenSpec: 規格中樞
- Antigravity: 後續實作者

5. 已定案 baseline

必做 baseline

- NewReno
- CUBIC

強烈建議 baseline

- BBR

加分 baseline

- Vegas
- Westwood
- DCTCP
- LEDBAT

6. 已定案 metrics

必做 metrics：

- average throughput
- average RTT
- packet loss rate
- utility score
- training reward curve
- convergence behavior

加分 metrics：

- queue occupancy
- Jain's fairness index
- generalization across different bandwidth / delay settings

7. 已定案 MVP

MVP 的最低完成標準是：

1. 明確定義 single bottleneck link 場景
2. 明確定義 NewReno / CUBIC / BBR baseline plan
3. 明確定義 ns3-gym environment 的 MDP 需求
4. 明確定義 DQN MVP 的 state / action / reward
5. 明確定義 throughput / RTT / loss / utility 評估方式
6. 後續能產出 baseline vs DRL 的比較圖表
7. GitHub README / PPT / 10 分鐘影片能清楚說明整體研究設計

二、你的任務角色

你現在不是一般助理，而是：

- OpenSpec 架構師
- 研究總設計師
- 技術 PM
- 論文 proposal 規劃顧問
- AI coding agent 工作規格設計者

你的產出必須足以讓後續 Antigravity 在不偏航的情況下，依照 OpenSpec change 逐步執行。

三、OpenSpec change 01 的核心目的

change-01-project-charter 的任務是建立整個專案的 **上游共識文件**。

它要回答：

1. 這個專題到底要做什麼？
2. 這個專題明確不做什麼？
3. 為什麼這題值得做？
4. 為什麼這題適合 DRL？
5. 為什麼要先從 single bottleneck link 開始？

6. 為什麼 baseline 要選 NewReno / CUBIC / BBR ?
7. 為什麼 MVP 先選 DQN 而不是直接 PP0 ?
8. 成功標準是什麼 ?
9. 失敗或卡關時如何降階 ?
10. 後續 change-02、03、04 應如何銜接 ?

四、你必須先搜尋並引用的來源

請你在正式輸出前先做最新網路搜尋。
不可只靠記憶。

優先使用來源

請優先搜尋並引用：

1. OpenSpec 官方網站 / GitHub README / docs
2. ns-3 官方 TCP model 文件
3. ns3-gym 官方 GitHub / ns-3 app 頁面
4. Stable-Baselines3 DQN 文件
5. Stable-Baselines3 PP0 文件
6. Aurora 原始論文
7. Pantheon paper / 官方 benchmark 說明
8. 必要時補充 RFC 或正式論文

你必須用來源確認的判斷

請確認並支撐以下判斷：

- OpenSpec 適合用來建立 spec-driven source of truth
- ns-3 適合做 TCP / congestion control baseline
- ns-3 支援 NewReno、CUBIC、BBR 等 TCP congestion control
- ns3-gym 適合將 ns-3 包成 RL environment
- DQN 適合 discrete action 的第一版 MVP
- PP0 適合留作 future extension, 不是本 change 必做
- Aurora 可支撐 DRL for congestion control 的研究正當性
- Pantheon 適合作為 benchmark / reproducibility 思想來源, 但不是 MVP 必裝工具
- throughput 不能是唯一成功指標, 必須同時納入 RTT / loss / utility

五、OpenSpec change 01 必須產出的內容

請嚴格按照以下結構輸出。

OpenSpec change 01: project-charter 總結

請用 150-250 字說明：

- change 01 的目的
- 為什麼它是整個專案的 source of truth
- 它如何約束後續 change 02 / 03 / 04
- 它如何避免 Antigravity 偏航

1. Change Metadata

請輸出：

- Change ID
- Change title

- Change type
- Status
- Owner
- Implementer
- Related phase
- Related downstream changes
- Required before coding: Yes / No
- Code implementation allowed in this change: Yes / No

請明確設定：

- Change ID: 01-project-charter
- Change type: documentation / governance / scope definition
- Status: draft for review
- Owner: user / spec owner
- Implementer: Antigravity, after approval
- Code implementation allowed: No

2. Project Charter

請產出正式 project charter, 包含：

2.1 Project Name

請包含：

- 中文名稱
- 英文名稱
- Repo name

2.2 Project Mission

用 1-2 段正式寫出這個專題的使命。

2.3 Project Background

請寫出：

- 網路吞吐量為什麼重要
- congestion control 為什麼是核心
- 為什麼這跟分散式系統 / P2P / IPFS 背景有知識連續性
- 為什麼本專題不直接做 IPFS

2.4 Problem Statement

請正式定義本專題要解的問題：

- 傳統 TCP congestion control 是重要 baseline
- 但 throughput / RTT / loss trade-off 不容易只靠單一指標處理
- 本研究將問題轉成 DRL sequential decision-making formulation

2.5 Proposed Direction

請說明：

- 使用 ns-3 建立 single bottleneck link
- 使用 ns3-gym 作為 RL interface
- 使用 SB3 DQN 作為 MVP agent
- 與 NewReno / CUBIC / BBR 比較
- 使用 throughput / RTT / loss / utility 評估

2.6 Success Definition

請明確定義什麼叫成功。

成功不等於：

- DRL 全面贏過所有 baseline
- 完成真實網路部署
- 完成 IPFS 實作

成功等於：

- 研究範圍明確
- baseline plan 明確
- MDP 明確
- metrics 明確
- experiment matrix 明確
- 後續 OpenSpec change 可執行
- final project 可形成 GitHub / PPT / video 交付物

3. Scope / Non-Scope

請以非常明確的方式列出。

3.1 In Scope

至少包含：

- single bottleneck link
- ns-3 simulation
- ns3-gym RL interface
- sender-side congestion-control abstraction
- DQN MVP
- NewReno / CUBIC / BBR baseline comparison
- throughput / RTT / loss / utility evaluation
- GitHub / PPT / 10-min video support
- OpenSpec-driven workflow

3.2 Out of Scope

至少包含：

- IPFS implementation
- QUIC implementation
- kernel-level TCP
- real Internet deployment
- multi-agent RL
- multi-path routing
- large-scale network topology
- Pantheon full integration as required dependency
- PPO as MVP requirement
- production-level congestion control protocol

3.3 Strict Non-Goals

請列出不能被 Antigravity 自行擴張的事項。

語氣要強硬、明確，例如：

- Do not implement IPFS.
- Do not replace the project topic.
- Do not jump to DRL before baseline benchmark is specified.
- Do not optimize for throughput alone.

4. Research Questions and Hypotheses

請把 Phase 1 的研究問題整理成 project-charter 版。

4.1 Main Research Question

請給一個正式主研究問題。

4.2 Sub-questions

至少包含：

- state representation
- action design
- reward design
- baseline comparison
- robustness / generalization

4.3 Hypotheses

請給 2–3 個 hypothesis。

語氣必須保守，不可寫成已證明結果。

5. Technical Direction Freeze

請正式凍結技術方向。

5.1 Simulator

ns-3

說明為什麼。

5.2 RL Interface

ns3-gym

說明為什麼。

5.3 Training Framework

Stable-Baselines3

說明為什麼。

5.4 MVP Algorithm

DQN

說明為什麼。

5.5 Future Algorithm

PPO

說明何時才導入。

5.6 Baselines

NewReno / CUBIC / BBR

說明每個 baseline 的角色。

5.7 Metrics

throughput / RTT / loss / utility / reward curve / convergence behavior

說明每個 metric 的用途。

6. MVP Definition

請明確定義 MVP。

6.1 MVP Must Have

列出必備項目。

6.2 MVP Should Have

列出強烈建議但非絕對必備項目。

6.3 MVP Could Have

列出加分項目。

6.4 MVP Must Not Have

列出禁止項目。

6.5 MVP Acceptance Criteria

請列出明確驗收條件。

注意：

這裡是 charter 層級，不要寫 code，也不要寫實際命令。

7. Downstream Change Map

請定義 change-01 後面三個 change 如何接續。

7.1 change-02-baseline-benchmark

請說明：

- 目的
- 輸入
- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

7.2 change-03-opengym-env

請說明：

- 目的
- 輸入
- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

7.3 change-04-drl-mvp

請說明：

- 目的
- 輸入
- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

8. Risk Register

請建立 charter 層級 risk register。

至少包含以下風險：

1. ns3-gym 安裝 / 版本問題

2. ns-3 baseline logging 不完整
3. BBR baseline 版本相依性
4. DQN 不收敛
5. Reward 設計導致高 throughput 但高 RTT / loss
6. Antigravity 自行擴題
7. 題目被誤解為 IPFS 實作
8. 結果沒有超越 baseline
9. 實驗圖表不足以支撐簡報
10. OpenSpec 文件與實作脫節

表格欄位請包含：

- Risk ID
- Description
- Probability
- Impact
- Prevention
- Fallback
- Owner
- Trigger condition

9. OpenSpec Artifact Blueprint

請設計 change-01-project-charter 應該產出的 artifact。

請不要真的建立檔案，但要提供建議結構與內容摘要。

至少包含：

```
```text
openspec/
 changes/
 01-project-charter/
 proposal.md
 design.md
 tasks.md
 specs/
 project-charter.md
 scope.md
 risk-register.md
```

對每個檔案說明：

- purpose
- required content
- reviewer should check
- acceptance criteria

## # 10. proposal.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/01-project-charter/proposal.md
```

的 markdown 草案。

內容至少包含：

- Why
- What Changes
- What Does Not Change
- Impact
- Dependencies
- Acceptance Criteria

注意：

這是 proposal.md 草案，不是正式論文 proposal。

---

## # 11. design.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/01-project-charter/design.md
```

的 markdown 草案。

內容至少包含：

- Design Goal
- Project Scope Boundary
- Technical Direction
- Governance Rules
- Downstream Change Dependency
- Decision Records

注意：

design.md 只處理設計與治理，不處理程式碼架構。

---

## # 12. tasks.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/01-project-charter/tasks.md
```

的 markdown 草案。

tasks 必須是文件任務，不是實作任務。

例如：

- Confirm project title
- Confirm scope / non-scope

- Confirm baseline list
- Confirm MVP success criteria
- Confirm risk register
- Review downstream changes

每個 task 請用 checkbox 格式。

---

## # 13. specs 草案摘要

請產出 specs 下三個檔案的摘要內容，不必寫成完整檔案，但要夠後續轉成完整 spec。

### 13.1 specs/project-charter.md

請提供：

- Purpose
- Project mission
- Research target
- Success definition
- OpenSpec governance role

### 13.2 specs/scope.md

請提供：

- In scope
- Out of scope
- Strict non-goals
- Expansion rules

### 13.3 specs/risk-register.md

請提供：

- Risk categories
  - Risk table summary
  - Fallback principles
- 

## # 14. Review Checklist

請建立 change-01 驗收檢查表。

檢查表必須讓我能回答：

- 題目是否凍結？
- scope 是否清楚？
- non-goals 是否清楚？
- baseline 是否凍結？
- metrics 是否凍結？
- MVP 是否凍結？
- 後續 change 是否能接上？
- 是否有任何內容偷偷進入實作？
- 是否有任何內容偏離 Phase 0 / Phase 1 定案？

請用 checklist 格式輸出。

## # 15. 給 Antigravity 的 change-01 指令草案

請幫我產出一段可以交給 Antigravity 的 change-01 指令。

這段指令必須說清楚：

- 目前只做 project charter
- 不准寫 code
- 不准建立 ns-3 實驗
- 不准開始 DQN
- 必須只產出 OpenSpec artifacts
- 必須遵守 scope / non-goals
- 完成後等待 spec owner 驗收

## 六、輸出品質要求

### 1. 語言

- 全文使用繁體中文
- 必要英文專有名詞保留英文
- 檔名、change ID、artifact 名稱保持英文

### 2. 深度

不要只列大綱。

要寫到我可以直接複製部分內容去建立 OpenSpec 文件。

### 3. 嚴謹度

所有重要判斷要有可信來源支撐。  
若是推論，請明確寫「這是基於來源的推論」。

### 4. 禁止事項

本階段禁止：

- 寫任何程式碼
- 寫 shell command
- 產生 ns-3 實作
- 產生 Python 訓練腳本
- 進入 baseline benchmark
- 進入 opengym-env
- 進入 drl-mvp
- 更改題目
- 擴張到 IPFS / QUIC / multi-agent / multi-path
- 宣稱已有實驗結果

## 七、現在開始執行

請先搜尋官方文件與正式來源，再輸出。

請直接從：

## OpenSpec change 01：project-charter 總結

開始。

這份是 **Phase 2 / OpenSpec change 01：project-charter** 的最佳 prompt。它的設計依據是：OpenSpec 的核心是先把規格變成 single source of truth，再進入實作；官方與相關文件也強調以 proposal / spec / design / tasks 這類 artifacts 讓人與 AI coding assistant 在寫 code 前先對齊。([GitHub][1])

同時，這份 prompt 會把本專題的技術邊界鎖定在 ns-3 / ns3-gym / DRL congestion control 的 Phase 0、Phase 1 定案上；ns-3 官方支援多種 TCP congestion control baseline，ns3-gym 則是為 ns-3 與 RL 整合而設計，這些都會被要求寫進 charter 作為後續 source of truth。([NSNAM][2])

請下一回合直接貼上以下整段：

```
`text
```

你現在要執行的是我 DRL Final Project 的 **\*\*Phase 2：用 OpenSpec 建立專案真相來源\*\*** 的第一階段：

```
OpenSpec change 01:project-charter
```

```
最高任務目標
```

你要基於前面已完成的 Phase 0 與 Phase 1 定案內容，替我產出一份可以作為整個專案 **\*\*source of truth\*\*** 的 OpenSpec-style project charter。

這不是 proposal 草稿，也不是一般摘要，而是後續所有 change 的上游規格母文件。

它必須能讓我、Antigravity、OpenSpec、後續 PPT / GitHub / 10 分鐘影片全部對齊同一個方向。

本階段只產出文件，不寫任何程式碼，不做實作，不啟動 ns-3，不啟動 ns3-gym，不訓練模型。

```

```

```
一、Phase 0 / Phase 1 已凍結內容
```

請你完整承接以下已定案內容，不得自行改題、擴題或改回其他題目。

```
1. 正式題目
```

```
中文題目
```

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

```
英文題目
```

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

```
簡報首頁版題目
```

DRL-Based Congestion Control over a Bottleneck Link

```
建議 GitHub repo name
```

drl-bottleneck-congestion-control

```

```

```
2. 研究定位
```

本專題是：

- networking systems 題
- DRL sequential decision-making 題
- congestion control / throughput optimization 題
- ns-3 / ns3-gym / Stable-Baselines3 專題
- baseline comparison 專題
- proposal-first / spec-first 專題

本專題不是：

- IPFS 實作題
- QUIC 實作題
- Linux kernel TCP 修改題
- 真實 Internet deployment 題
- multi-agent 題
- multi-path routing 題
- 大型網路拓模題
- Pantheon full integration 題
- 自適應交通號誌控制題

```

```

```
3. 核心研究主張
```

本研究將單一瓶頸鏈路的壅塞控制建模為深度強化學習問題，讓 agent 根據網路狀態觀測值調整傳輸控制行為，並透過 throughput、RTT、packet loss 與 utility 指標，和 NewReno、CUBIC、BBR 等 TCP baseline 進行比較。

本研究不承諾 DRL 必然全面超越所有 TCP baseline。

成功標準是建立清楚的 DRL formulation、可重現的 baseline / experiment plan、以及能支撐後續實作的 OpenSpec source of truth。

---

## ## 4. 已定案工具鏈

後續 implementation 預設工具鏈為：

- ns-3: network simulator
- ns3-gym: ns-3 與 RL agent 的介面
- Stable-Baselines3: DRL 訓練框架
- DQN: MVP 第一版演算法，因為 action 初始設計為 discrete
- PPO: future extension, 不是 MVP 必做
- OpenSpec: 規格中樞
- Antigravity: 後續實作者

---

## ## 5. 已定案 baseline

### ### 必做 baseline

- NewReno
- CUBIC

### ### 強烈建議 baseline

- BBR

### ### 加分 baseline

- Vegas
- Westwood
- DCTCP
- LEDBAT

---

## ## 6. 已定案 metrics

必做 metrics：

- average throughput
- average RTT
- packet loss rate
- utility score
- training reward curve
- convergence behavior

加分 metrics：

- queue occupancy
- Jain's fairness index
- generalization across different bandwidth / delay settings

---

## ## 7. 已定案 MVP

MVP 的最低完成標準是：

1. 明確定義 single bottleneck link 場景
2. 明確定義 NewReno / CUBIC / BBR baseline plan
3. 明確定義 ns3-gym environment 的 MDP 需求
4. 明確定義 DQN MVP 的 state / action / reward

5. 明確定義 throughput / RTT / loss / utility 評估方式
6. 後續能產出 baseline vs DRL 的比較圖表
7. GitHub README / PPT / 10 分鐘影片能清楚說明整體研究設計

---

## # 二、你的任務角色

你現在不是一般助理，而是：

- OpenSpec 架構師
- 研究總設計師
- 技術 PM
- 論文 proposal 規劃顧問
- AI coding agent 工作規格設計者

你的產出必須足以讓後續 AntigraVity 在不偏航的情況下，依照 OpenSpec change 逐步執行。

---

## # 三、OpenSpec change 01 的核心目的

change-01-project-charter 的任務是建立整個專案的 **\*\*上游共識文件\*\***。

它要回答：

1. 這個專題到底要做什麼？
2. 這個專題明確不做什麼？
3. 為什麼這題值得做？
4. 為什麼這題適合 DRL？
5. 為什麼要先從 single bottleneck link 開始？
6. 為什麼 baseline 要選 NewReno / CUBIC / BBR？
7. 為什麼 MVP 先選 DQN 而不是直接 PPO？
8. 成功標準是什麼？
9. 失敗或卡關時如何降階？
10. 後續 change-02、03、04 應如何銜接？

---

## # 四、你必須先搜尋並引用的來源

請你在正式輸出前先用最新網路搜尋。

不可只靠記憶。

### ## 優先使用來源

請優先搜尋並引用：

1. OpenSpec 官方網站 / GitHub README / docs
2. ns-3 官方 TCP model 文件
3. ns3-gym 官方 GitHub / ns-3 app 頁面
4. Stable-Baselines3 DQN 文件
5. Stable-Baselines3 PPO 文件
6. Aurora 原始論文
7. Pantheon paper / 官方 benchmark 說明
8. 必要時補充 RFC 或正式論文

### ## 你必須用來源確認的判斷

請確認並支撐以下判斷：

- OpenSpec 適合用來建立 spec-driven source of truth
- ns-3 適合做 TCP / congestion control baseline

- ns-3 支援 NewReno、CUBIC、BBR 等 TCP congestion control
- ns3-gym 適合將 ns-3 包成 RL environment
- DQN 適合 discrete action 的第一版 MVP
- PPO 適合留作 future extension, 不是本 change 必做
- Aurora 可支撐 DRL for congestion control 的研究正當性
- Pantheon 適合作為 benchmark / reproducibility 思想來源, 但不是 MVP 必裝工具
- throughput 不能是唯一成功指標, 必須同時納入 RTT / loss / utility

---

## # 五、OpenSpec change 01 必須產出的內容

請嚴格按照以下結構輸出。

---

### ## # OpenSpec change 01: project-charter 總結

請用 150-250 字說明：

- change 01 的目的
- 為什麼它是整個專案的 source of truth
- 它如何約束後續 change 02 / 03 / 04
- 它如何避免 Antigravity 偏航

---

### ## # 1. Change Metadata

請輸出：

- Change ID
- Change title
- Change type
- Status
- Owner
- Implementer
- Related phase
- Related downstream changes
- Required before coding: Yes / No
- Code implementation allowed in this change: Yes / No

請明確設定：

- Change ID: 01-project-charter
- Change type: documentation / governance / scope definition
- Status: draft for review
- Owner: user / spec owner
- Implementer: Antigravity, after approval
- Code implementation allowed: No

---

### ## # 2. Project Charter

請產出正式 project charter, 包含：

#### ### 2.1 Project Name

請包含：

- 中文名稱
- 英文名稱
- Repo name

#### ### 2.2 Project Mission

用 1-2 段正式寫出這個專題的使命。

### ### 2.3 Project Background

請寫出：

- 網路吞吐量為什麼重要
- congestion control 為什麼是核心
- 為什麼這跟分散式系統 / P2P / IPFS 背景有知識連續性
- 為什麼本專題不直接做 IPFS

### ### 2.4 Problem Statement

請正式定義本專題要解的問題：

- 傳統 TCP congestion control 是重要 baseline
- 但 throughput / RTT / loss trade-off 不容易只靠單一指標處理
- 本研究將問題轉成 DRL sequential decision-making formulation

### ### 2.5 Proposed Direction

請說明：

- 使用 ns-3 建立 single bottleneck link
- 使用 ns3-gym 作為 RL interface
- 使用 SB3 DQN 作為 MVP agent
- 與 NewReno / CUBIC / BBR 比較
- 使用 throughput / RTT / loss / utility 評估

### ### 2.6 Success Definition

請明確定義什麼叫成功。

成功不等於：

- DRL 全面贏過所有 baseline
- 完成真實網路部署
- 完成 IPFS 實作

成功等於：

- 研究範圍明確
- baseline plan 明確
- MDP 明確
- metrics 明確
- experiment matrix 明確
- 後續 OpenSpec change 可執行
- final project 可形成 GitHub / PPT / video 交付物

---

## ## # 3. Scope / Non-Scope

請以非常明確的方式列出。

### ### 3.1 In Scope

至少包含：

- single bottleneck link
- ns-3 simulation
- ns3-gym RL interface
- sender-side congestion-control abstraction
- DQN MVP
- NewReno / CUBIC / BBR baseline comparison
- throughput / RTT / loss / utility evaluation

- GitHub / PPT / 10-min video support
- OpenSpec-driven workflow

### ### 3.2 Out of Scope

至少包含：

- IPFS implementation
- QUIC implementation
- kernel-level TCP
- real Internet deployment
- multi-agent RL
- multi-path routing
- large-scale network topology
- Pantheon full integration as required dependency
- PPO as MVP requirement
- production-level congestion control protocol

### ### 3.3 Strict Non-Goals

請列出不能被 Antigravity 自行擴張的事項。

語氣要強硬、明確，例如：

- Do not implement IPFS.
- Do not replace the project topic.
- Do not jump to DRL before baseline benchmark is specified.
- Do not optimize for throughput alone.

---

## ## # 4. Research Questions and Hypotheses

請把 Phase 1 的研究問題整理成 project-charter 版。

### ### 4.1 Main Research Question

請給一個正式主研究問題。

### ### 4.2 Sub-questions

至少包含：

- state representation
- action design
- reward design
- baseline comparison
- robustness / generalization

### ### 4.3 Hypotheses

請給 2-3 個 hypothesis。

語氣必須保守，不可寫成已證明結果。

---

## ## # 5. Technical Direction Freeze

請正式凍結技術方向。

### ### 5.1 Simulator

ns-3

說明為什麼。

### ### 5.2 RL Interface

ns3-gym

說明為什麼。

### ### 5.3 Training Framework

## Stable-Baselines3

說明為什麼。

### ### 5.4 MVP Algorithm

#### DQN

說明為什麼。

### ### 5.5 Future Algorithm

#### PPO

說明何時才導入。

### ### 5.6 Baselines

NewReno / CUBIC / BBR

說明每個 baseline 的角色。

### ### 5.7 Metrics

throughput / RTT / loss / utility / reward curve / convergence behavior

說明每個 metric 的用途。

---

## ## # 6. MVP Definition

請明確定義 MVP。

### ### 6.1 MVP Must Have

列出必備項目。

### ### 6.2 MVP Should Have

列出強烈建議但非絕對必備項目。

### ### 6.3 MVP Could Have

列出加分項目。

### ### 6.4 MVP Must Not Have

列出禁止項目。

### ### 6.5 MVP Acceptance Criteria

請列出明確驗收條件。

注意：

這裡是 charter 層級，不要寫 code，也不要寫實際命令。

---

## ## # 7. Downstream Change Map

請定義 change-01 後面三個 change 如何接續。

### ### 7.1 change-02-baseline-benchmark

請說明：

- 目的
- 輸入

- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

### ### 7.2 change-03-opengym-env

請說明：

- 目的
- 輸入
- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

### ### 7.3 change-04-drl-mvp

請說明：

- 目的
- 輸入
- 輸出
- 成功標準
- 依賴 change-01 的哪些內容

---

## ## # 8. Risk Register

請建立 charter 層級 risk register。

至少包含以下風險：

1. ns3-gym 安裝 / 版本問題
2. ns-3 baseline logging 不完整
3. BBR baseline 版本相依性
4. DQN 不收斂
5. Reward 設計導致高 throughput 但高 RTT / loss
6. Antigravity 自行擴題
7. 題目被誤解為 IPFS 實作
8. 結果沒有超越 baseline
9. 實驗圖表不足以支撐簡報
10. OpenSpec 文件與實作脫節

表格欄位請包含：

- Risk ID
- Description
- Probability
- Impact
- Prevention
- Fallback
- Owner
- Trigger condition

---

## ## # 9. OpenSpec Artifact Blueprint

請設計 change-01-project-charter 應該產出的 artifact。

請不要真的建立檔案，但要提供建議結構與內容摘要。

至少包含：

```
text
openspec/
changes/
 01-project-charter/
 proposal.md
 design.md
 tasks.md
 specs/
 project-charter.md
 scope.md
 risk-register.md
、
```

對每個檔案說明：

- \* purpose
- \* required content
- \* reviewer should check
- \* acceptance criteria

---

## # 10. proposal.md 草案內容

請直接產出一份可以放入：

```
text
openspec/changes/01-project-charter/proposal.md
```

的 markdown 草案。

內容至少包含：

- \* Why
- \* What Changes
- \* What Does Not Change
- \* Impact
- \* Dependencies
- \* Acceptance Criteria

注意：

這是 proposal.md 草案，不是正式論文 proposal。

---

## # 11. design.md 草案內容

請直接產出一份可以放入：

```
text
openspec/changes/01-project-charter/design.md
```

的 markdown 草案。

內容至少包含：

- \* Design Goal
- \* Project Scope Boundary
- \* Technical Direction
- \* Governance Rules
- \* Downstream Change Dependency
- \* Decision Records

注意：

design.md 只處理設計與治理，不處理程式碼架構。

---

## ## # 12. tasks.md 草案內容

請直接產出一份可以放入：

text

openspec/changes/01-project-charter/tasks.md

的 markdown 草案。

tasks 必須是文件任務，不是實作任務。

例如：

- \* Confirm project title
- \* Confirm scope / non-scope
- \* Confirm baseline list
- \* Confirm MVP success criteria
- \* Confirm risk register
- \* Review downstream changes

每個 task 請用 checkbox 格式。

---

## ## # 13. specs 草案摘要

請產出 specs 下三個檔案的摘要內容，不必寫成完整檔案，但要夠後續轉成完整 spec。

### ### 13.1 specs/project-charter.md

請提供：

- \* Purpose
- \* Project mission
- \* Research target
- \* Success definition
- \* OpenSpec governance role

### ### 13.2 specs/scope.md

請提供：

- \* In scope
- \* Out of scope
- \* Strict non-goals
- \* Expansion rules

### ### 13.3 specs/risk-register.md

請提供：

- \* Risk categories
- \* Risk table summary
- \* Fallback principles

---

## ## # 14. Review Checklist

請建立 change-01 驗收檢查表。

檢查表必須讓我能回答：

- \* 題目是否凍結？

- \* scope 是否清楚？
- \* non-goals 是否清楚？
- \* baseline 是否凍結？
- \* metrics 是否凍結？
- \* MVP 是否凍結？
- \* 後續 change 是否能接上？
- \* 是否有任何內容偷偷進入實作？
- \* 是否有任何內容偏離 Phase 0 / Phase 1 定案？

請用 checklist 格式輸出。

---

## ## # 15. 給 Antigravity 的 change-01 指令草案

請幫我產出一段可以交給 Antigravity 的 change-01 指令。

這段指令必須說清楚：

- \* 目前只做 project charter
- \* 不准寫 code
- \* 不准建立 ns-3 實驗
- \* 不准開始 DQN
- \* 必須只產出 OpenSpec artifacts
- \* 必須遵守 scope / non-goals
- \* 完成後等待 spec owner 驗收

---

## # 六、輸出品質要求

### ## 1. 語言

- \* 全文使用繁體中文
- \* 必要英文專有名詞保留英文
- \* 檔名、change ID、artifact 名稱保持英文

### ## 2. 深度

不要只列大綱。

要寫到我可以直接複製部分內容去建立 OpenSpec 文件。

### ## 3. 嚴謹度

所有重要判斷要有可信來源支撐。

若是推論，請明確寫「這是基於來源的推論」。

### ## 4. 禁止事項

本階段禁止：

- \* 寫任何程式碼
- \* 寫 shell command
- \* 產生 ns-3 實作
- \* 產生 Python 訓練腳本
- \* 進入 baseline benchmark
- \* 進入 opengym-env
- \* 進入 drl-mvp
- \* 更改題目
- \* 擴張到 IPFS / QUIC / multi-agent / multi-path
- \* 宣稱已有實驗結果

---

## # 七、現在開始執行

請先搜尋官方文件與正式來源，再輸出。

請直接從：

# OpenSpec change 01: project-charter 總結

開始。

```
::contentReference[oaicite:2]{index=2}
```

[1]: [https://github.com/Fission-AI/OpenSpec?utm\\_source=chatgpt.com](https://github.com/Fission-AI/OpenSpec?utm_source=chatgpt.com) "Fission-AI/OpenSpec: Spec-driven development (SDD) ..."

[2]: [https://www.nsnam.org/docs/models/html/tcp.html?utm\\_source=chatgpt.com](https://www.nsnam.org/docs/models/html/tcp.html?utm_source=chatgpt.com) "16.6. TCP models in ns-3"

顯示更多顯示較少

# OpenSpec change 01 : project-charter 總結

**change-01-project-charter** 的目的，是把本專題的題目、研究邊界、成功標準、工具鏈、baseline、風險與後續 change 串接關係，正式寫成整個專案的 **source of truth**。OpenSpec 官方強調在寫 code 前，讓人類與 AI 先對齊 specs；每個 change 也可用資料夾集中放置 proposal、design、tasks、specs，避免規格散落在聊天紀錄或臨時筆記中。 [GitHub+1](#)

因此，change 01 不允許寫 code，也不允許啟動 ns-3 / ns3-gym / DQN。它只負責建立治理文件，約束後續 **change-02-baseline-benchmark**、**change-03-opengym-env**、**change-04-drl-mvp** 的方向，避免 Antigravity 自行擴題、跳過 baseline、或把專題誤改成 IPFS / QUIC / multi-agent 題。

## 1. Change Metadata

欄位	內容
Change ID	<b>01-project-charter</b>
Change title	Project Charter for DRL-Based Congestion Control over a Bottleneck Link
Change type	documentation / governance / scope definition
Status	draft for review
Owner	user / spec owner
Implementer	Antigravity, after approval
Related phase	Phase 2：用 OpenSpec 建立專案真相來源
Related downstream changes	<b>02-ns3-baseline-benchmark, 03-opengym-env, 04-drl-mvp</b>
Required before coding	Yes
Code implementation allowed in this change	No

### 治理判斷：

此 change 是所有後續 change 的前置條件。只要 change 01 尚未被 spec owner 驗收，Antigravity 不得建立 ns-3 實驗、不准串 ns3-gym、不准寫 DQN、不准建立任何實作檔案。

## 2. Project Charter

### 2.1 Project Name

類型	名稱
中文名稱	以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化
英文名稱	Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link
簡報首頁版	DRL-Based Congestion Control over a Bottleneck Link
Repo name	<b>drl-bottleneck-congestion-control</b>

### 2.2 Project Mission

本專題的使命，是將單一瓶頸鏈路上的壅塞控制問題，形式化為一個可被深度強化學習處理的序列決策問題。專題將以 **throughput**、**RTT**、**packet loss**、**utility** 作為核心評估指標，規劃一個可重現、可比較、可交付的 DRL congestion-control prototype 路線。

本專題的重點不是宣稱 DRL 必然全面優於傳統 TCP，而是建立一套清楚的研究設計：先用 ns-3 規劃 single bottleneck link 與 TCP baseline，再透過 ns3-gym 將網路環境包成 RL environment，最後以 DQN MVP 作為第一階段 DRL agent。ns-3 官方 TCP model 支援多種 TCP congestion control 演算法，包含 CUBIC、NewReno、BBR 等，適合作為 baseline planning 的基礎。 

### 2.3 Project Background

網路吞吐量是分散式系統、內容傳輸、P2P 網路與區塊鏈網路協定中的基礎效能問題。對 IPFS 這類內容定址與 P2P 傳輸系統而言，真正的資料可用性與使用體驗，不只取決於內容是否被找到，也取決於底層網路傳輸是否能在瓶頸、延遲與丟包環境下維持合理效能。

壅塞控制是影響 throughput、RTT 與 packet loss 的核心機制。傳統 TCP congestion control 已非常成熟，但它們通常以固定規則或啟發式控制邏輯回應網路狀態。本專題的研究動機，是將壅塞控制改寫為 DRL agent 的 sequential decision-making task，讓 agent 依據觀測到的網路狀態做控制決策。Aurora 論文已指出，將 congestion control cast 為 RL 可以訓練 deep network policy 以捕捉資料流與網路條件中的複雜模式，同時也

 Proceedings of  
Machine Learning  
Research

提醒真實部署仍面臨 fairness、safety、generalization 等挑戰。

本專題與使用者的 IPFS / 區塊鏈網路協定畢論方向具有知識連續性，但本專題**不直接實作 IPFS**。這是刻意的 scope control：先把底層壅塞控制與吞吐量最佳化問題做小、做清楚、做成可交付成果，再把 IPFS 作為 future extension 或研究動機，不讓 final project 被 P2P overlay、DHT、Bitswap、實網節點整合拖垮。

## 2.4 Problem Statement

本專題要解的問題是：

如何在單一瓶頸鏈路中，將壅塞控制建模為深度強化學習問題，讓 agent 根據網路狀態調整傳輸控制行為，並與 NewReno、CUBIC、BBR 等 TCP baseline 比較 throughput、RTT、loss 與 utility 的 trade-off？

傳統 TCP congestion control 是本題的重要 baseline，而不是被否定的對象。NewReno、CUBIC、BBR 代表不同設計邏輯的壅塞控制方法；其中 ns-3 官方文件列出 native ns-3 TCP model 支援 CUBIC、NewReno、BBR 等演算法，因此它們適合作為後續 baseline benchmark 的規格依據。  [NSNAM](#)

問題的核心不只是「提高 throughput」。如果只最佳化 throughput，可能導致 queue 累積、RTT 增加、packet loss 上升，甚至形成 bufferbloat。因此，本專題將 throughput、RTT、loss 與 utility 一起納入評估，並在 reward 設計中明確避免 throughput-only 的錯誤方向。這是基於 Aurora 與 congestion-control

 [Proceedings of Machine Learning Research](#)

benchmark 研究對真實採用、可比較性與多指標權衡的提醒所做的設計推論。 [Learning Research+1](#)

## 2.5 Proposed Direction

本專題的 proposed direction 正式凍結如下：

### 1. 使用 ns-3 建立 single bottleneck link 場景

ns-3 是 Internet systems 的 discrete-event network simulator，主要用於 research and education；其 TCP model 支援多種 congestion control 演算法，適合做 baseline planning。  [NSNAM+1](#)

### 2. 使用 ns3-gym 作為 RL interface

ns3-gym 的定位是整合 OpenAI Gym 與 ns-3，以支援 networking research 中的 RL workflow。

 [GitHub+1](#)

### 3. 使用 Stable-Baselines3 作為訓練框架

Stable-Baselines3 是 PyTorch 上可靠的 RL algorithm implementations；其 DQN 文件明確標示 action space 為 Discrete，適合 MVP 的離散 action 設計。  [GitHub+1](#)

### 4. 使用 DQN 作為 MVP agent

這是基於來源的推論：因為 MVP action 預設為 decrease / keep / increase 這種 discrete action，而 SB3

 [stable-baselines3](#)

DQN 對應 Discrete action space，因此 DQN 比 PPO 更適合第一版 MVP。 [baselines3.readthedocs.io+1](#)

### 5. PPO 作為 future extension

PPO 支援更一般的 policy optimization，SB3 文件也說明 PPO 使用 clipping 避免 policy update 過大；但

 [stable-baselines3](#)

它不是 change 01 或 MVP 必做項目。 [baselines3.readthedocs.io+1](#)

### 6. 與 NewReno / CUBIC / BBR 比較

這三者構成 baseline 核心，支撐後續 baseline benchmark change。

### 7. 使用 throughput / RTT / loss / utility 評估

不以單一 throughput 作為成功標準。

## 2.6 Success Definition

### 成功不等於

- DRL 全面贏過所有 baseline
- 完成真實網路部署
- 完成 IPFS 實作
- 完成 QUIC 協定實作
- 寫出 production-level congestion control protocol
- 完成 multi-agent 或 multi-path 系統

### 成功等於

- 題目、scope、non-goals 明確凍結
- baseline plan 明確
- MDP formulation 明確
- metrics 明確
- experiment matrix 明確
- 後續 OpenSpec changes 可執行
- Antigravity 能依規格逐步工作且不偏航
- 最終能支撐 GitHub / PPT / 10 分鐘影片交付物
- 若 DRL 未超越 BBR / CUBIC，也能以 trade-off analysis、limitation、risk control 作為合理 final finding

## 3. Scope / Non-Scope

### 3.1 In Scope

本專題 scope 正式包含：

- single bottleneck link
- ns-3 simulation
- ns3-gym RL interface
- sender-side congestion-control abstraction
- DQN MVP
- NewReno / CUBIC / BBR baseline comparison
- throughput / RTT / loss / utility evaluation
- GitHub / PPT / 10-min video support

- OpenSpec-driven workflow
- baseline-first implementation strategy
- proposal-first / spec-first 專案治理

## 3.2 Out of Scope

本專題 scope 正式排除：

- IPFS implementation
- QUIC implementation
- kernel-level TCP
- real Internet deployment
- multi-agent RL
- multi-path routing
- large-scale network topology
- Pantheon full integration as required dependency
- PPO as MVP requirement
- production-level congestion control protocol
- full Aurora reproduction
- blockchain network simulation
- libp2p / Bitswap / DHT modification

## 3.3 Strict Non-Goals

以下事項不得由 Antigravity 自行擴張：

- **Do not implement IPFS.**
- **Do not implement QUIC.**
- **Do not modify Linux kernel TCP.**
- **Do not replace the project topic.**
- **Do not jump to DRL before baseline benchmark is specified.**
- **Do not optimize for throughput alone.**
- **Do not make Pantheon a required dependency.**
- **Do not introduce multi-agent RL.**
- **Do not introduce multi-path routing.**
- **Do not create large-scale topology.**
- **Do not claim experimental results before experiments exist.**

- Do not treat PPO as an MVP requirement.

## 4. Research Questions and Hypotheses

### 4.1 Main Research Question

**How can congestion control over a single bottleneck link be formulated as a deep reinforcement learning problem to balance throughput, RTT, and packet loss compared with traditional TCP baselines?**

中文：

如何將單一瓶頸鏈路壅塞控制形式化為深度強化學習問題，並與傳統 TCP baseline 比較其在 throughput、RTT 與 packet loss 之間的權衡能力？

### 4.2 Sub-questions

#### 1. State representation

哪些網路觀測值最適合描述壅塞控制 agent 的 state，例如 recent throughput、RTT、loss、queue / congestion indicator、previous action？

#### 2. Action design

MVP 階段是否能用離散 action，例如 decrease / keep / increase sending rate 或 cwnd-like control，形成可訓練的第一版 agent？

#### 3. Reward design

如何設計 reward，避免 agent 只追求 throughput，而忽略 RTT 與 packet loss？

#### 4. Baseline comparison

DRL agent 與 NewReno、CUBIC、BBR 相比，能呈現哪些 throughput-delay-loss trade-off？

#### 5. Robustness / generalization

在不同 bandwidth、delay、queue 或 cross traffic 條件下，agent 的策略是否仍能維持穩定表現？

### 4.3 Hypotheses

#### H1

將 throughput、RTT、packet loss 同時納入 reward，可能比 throughput-only reward 更能避免高延遲或高丟包策略。

#### H2

DQN 搭配離散 action 可以作為第一版 DRL congestion-control prototype 的最低風險路線，但不保證全面超越 CUBIC 或 BBR。

#### H3

在變動 bandwidth 或存在 cross traffic 的場景中，DRL agent 可能展現可分析的 adaptive behavior；但此假設需由後續實驗驗證，change 01 不宣稱結果。

## 5. Technical Direction Freeze

### 5.1 Simulator：ns-3

正式凍結：ns-3。

理由：ns-3 是用於 Internet systems 的 discrete-event network simulator，主要面向 research and educational use；其 TCP model 支援多種 congestion control 演算法，適合作為 baseline 實驗規劃基礎。



### 5.2 RL Interface：ns3-gym

正式凍結：ns3-gym。

理由：ns3-gym 的目的就是將 OpenAI Gym 與 ns-3 整合，以支援 reinforcement learning in networking research。這是本題從 ns-3 simulation 接到 Python RL agent 的主要橋接層。  [GitHub+1](#)

### 5.3 Training Framework：Stable-Baselines3

正式凍結：Stable-Baselines3。

理由：SB3 是 PyTorch 上可靠的 RL algorithm implementations，並提供 DQN、PPO 等演算法文件與一致化 API，適合作為後續 DRL MVP 的訓練框架。  [GitHub](#)

### 5.4 MVP Algorithm：DQN

正式凍結：DQN。

理由：MVP action 設計為 discrete action，例如 decrease / keep / increase。SB3 DQN 文件明確將 action space 標示為 Discrete，因此 DQN 是第一版最小可行 DRL agent 的合理選擇。  [stable-baselines3.readthedocs.io](#)

### 5.5 Future Algorithm：PPO

正式凍結：PPO 為 future extension，不是 MVP 必做。

PPO 支援更一般的 policy optimization，SB3 文件說明 PPO 透過 clipping 避免 policy update 過大；但它應在 DQN MVP、baseline logging、ns3-gym smoke test 成功後再考慮。  [stable-baselines3.readthedocs.io](#)

### 5.6 Baselines：NewReno / CUBIC / BBR

Baseline	角色
NewReno	經典 loss-based TCP baseline
CUBIC	現代常見 TCP congestion control baseline ; ns-3 文件列為支援演算法且指出 CUBIC 為 default
BBR	以 bottleneck bandwidth / RTT 思維形成不同比較視角

ns-3 官方 TCP 文件列出 NewReno、CUBIC、BBR 等支援演算法，因此三者適合作為本專題 baseline freeze。



## 5.7 Metrics

Metric	用途
throughput	衡量成功資料傳輸效率
RTT	衡量延遲與 queue 累積情況
packet loss	衡量壅塞嚴重程度
utility	綜合 throughput、RTT、loss 的整體分數
reward curve	觀察 DRL 訓練過程
convergence behavior	評估 agent 是否學到穩定策略

## 6. MVP Definition

### 6.1 MVP Must Have

MVP 必須包含：

- 明確定義 single bottleneck link 場景
- 明確定義 baseline plan：NewReno、CUBIC；BBR 強烈建議
- 明確定義 ns3-gym environment 的 MDP 需求
- 明確定義 DQN MVP 的 state / action / reward
- 明確定義 throughput / RTT / loss / utility 評估方式
- 明確定義 experiment matrix 的最低版本
- 明確產生 GitHub / PPT / 10-min video 可用的研究敘事
- 明確產出後續 change-02、change-03、change-04 的銜接規格

### 6.2 MVP Should Have

MVP 強烈建議包含：

- BBR baseline

- utility score formula
- reward normalization 設計說明
- variable bandwidth scenario
- cross traffic scenario
- risk fallback plan
- baseline vs DRL 圖表規格

## 6.3 MVP Could Have

MVP 加分項目：

- PPO comparison
- queue occupancy
- Jain's fairness index
- multi-flow shared bottleneck
- reward ablation
- generalization across bandwidth / delay settings

## 6.4 MVP Must Not Have

MVP 禁止包含：

- IPFS implementation
- QUIC implementation
- Linux kernel TCP modification
- production deployment
- multi-agent RL
- multi-path routing
- large-scale topology
- Pantheon full integration as required dependency
- PPO as required algorithm
- direct claim that DRL outperforms all baselines

## 6.5 MVP Acceptance Criteria

MVP charter 層級驗收條件：

- ☐ 題目與 repo name 已凍結。
- ☐ In scope / out of scope / strict non-goals 已明確。

- ☐ Baselines 已凍結為 NewReno、CUBIC，BBR 強烈建議。
- ☐ Metrics 已凍結為 throughput、RTT、loss、utility、reward curve、convergence behavior。
- ☐ MVP algorithm 已凍結為 DQN。
- ☐ PPO 已明確標示為 future extension。
- ☐ 成功標準不以 DRL 全面勝過 baseline 為唯一標準。
- ☐ 後續 change-02 / 03 / 04 可從本 charter 接續。
- ☐ 本 change 未包含任何程式碼、命令或實作內容。

## 7. Downstream Change Map

### 7.1 change-02-baseline-benchmark

#### 目的

建立 ns-3 single bottleneck link baseline benchmark 規格。

#### 輸入

- change-01 的 project scope
- baseline list：NewReno、CUBIC、BBR
- metrics：throughput、RTT、loss、utility
- non-goals：不做 DRL、不做 IPFS、不做 QUIC

#### 輸出

- baseline benchmark specification
- topology specification
- logging requirements
- metric definition
- baseline acceptance criteria

#### 成功標準

- baseline 場景清楚
- 至少 NewReno / CUBIC 可作為第一批 benchmark
- BBR 作為強烈建議或加分
- 指標輸出需求明確
- 不進入 DRL 訓練

#### 依賴 change-01 的內容

- 題目
- scope
- non-goals
- baseline freeze
- metrics freeze

## 7.2 change-03-opengym-env

### 目的

定義 ns3-gym environment 的 MDP interface 規格。

### 輸入

- change-01 的 MDP 方向
- state / action / reward freeze
- ns-3 topology boundary
- DQN discrete action requirement

### 輸出

- observation space specification
- action space specification
- reward function specification
- episode / reset / step conceptual design
- random agent smoke test acceptance criteria

### 成功標準

- state / action / reward 可由 Antigravity 實作
- environment 不擴題到 IPFS / QUIC / multi-agent
- reward 不只最佳化 throughput
- action 保持 discrete MVP

### 依賴 change-01 的內容

- technical direction freeze
- DQN MVP decision
- reward philosophy
- scope / strict non-goals

## 7.3 change-04-drl-mvp

### 目的

定義 DQN MVP agent 的訓練與評估規格。

### 輸入

- change-03 的 environment spec
- change-02 的 baseline metrics
- DQN as MVP algorithm
- evaluation philosophy

### 輸出

- DQN MVP specification
- training / evaluation artifact requirements
- comparison table requirements
- reward curve / metric chart requirements

### 成功標準

- DQN agent 能被明確驗收
- 比較對象與 metrics 一致
- 不宣稱必然超越 baseline
- 產出可支撐 PPT / README / video 的圖表規格

### 依賴 change-01 的內容

- MVP definition
- baseline freeze
- metric freeze
- success definition
- risk register

## 8. Risk Register

Risk ID	Description	Probability	Impact	Prevention	Fallback	Owner	Trigger condition
R1	ns3-gym 安裝 / 版本問題	High	High	change-03 前固定 OS、ns-3、Python、ns3-gym 版本	改用 simplified Python Gym bottleneck	Antigravity	ns3-gym example 無法完成 smoke test

Risk ID	Description	Probability	Impact	Prevention	Fallback	Owner	Trigger condition
					environment 作 fallback		
R2	ns-3 baseline logging 不完整	Medium	High	change-02 先定義 throughput / RTT / loss logging requirements	先保留 throughput + RTT, loss 作 should-have	Antigravity	baseline 圖表缺少核心 metric
R3	BBR baseline 版本相依性	Medium	Medium	BBR 列為 strongly recommended, 不是 MVP hard requirement	MVP 先 NewReno + CUBIC, BBR 延後	Antigravity	BBR 在指定 ns-3 版本中不可用或不穩
R4	DQN 不收斂	Medium	High	使用 minimal state、discrete action、清楚 reward	改成 DQN behavior analysis, 不宣稱超越	Antigravity	reward curve 無穩定趨勢
R5	Reward 導致高 throughput 但高 RTT / loss	High	High	reward 必須含 RTT / loss penalty	做 throughput-only vs trade-off reward ablation	Spec owner + Antigravity	throughput 上升但 RTT / loss 異常
R6	Antigravity 自行擴題	Medium	High	OpenSpec change-by-change ; strict non-goals 寫入 charter	停止實作, 回到 change review	Spec owner	出現 IPFS / QUIC / multi-agent / multi-path 實作
R7	題目被誤解為 IPFS 實作	Medium	Medium	摘要與 charter 明確寫 IPFS only as motivation	PPT / README 移除主標題中的 IPFS	Spec owner	評審或文件將主題理解成 IPFS
R8	結果沒有超越 baseline	High	Medium	成功標準改為 trade-off analysis 與 reproducible workflow	final report 強調 limitation 與 scenario-level finding	Spec owner	DQN 指標落後於 CUBIC / BBR
R9	實驗圖表不足以支撐簡報	Medium	Medium	change-05 提前定義 required figures	保留 baseline-only + DRL prototype architecture 作報告	Antigravity	PPT 缺 throughput / RTT / loss 圖
R10	OpenSpec 文件與實作脫節	Medium	High	每個 change 先驗收 artifacts 再實作	暫停實作, 更新 spec 後再繼續	Spec owner	實作產物不符合 approved change

## 9. OpenSpec Artifact Blueprint

建議 change-01 artifact 結構：

```
openspec/
 changes/
 01-project-charter/
 proposal.md
 design.md
 tasks.md
 specs/
 project-charter.md
 scope.md
 risk-register.md
```

### proposal.md

項目	內容
purpose	說明為什麼需要建立 project charter
required content	Why, What Changes, What Does Not Change, Impact, Dependencies, Acceptance Criteria
reviewer should check	是否明確凍結題目、scope、non-goals、baseline、metrics
acceptance criteria	不含 code、不含實作命令、可作為後續 change 上游依據

### design.md

項目	內容
purpose	定義專案治理與設計決策
required content	Design Goal, Scope Boundary, Technical Direction, Governance Rules, Downstream Dependency, Decision Records
reviewer should check	是否防止 Antigravity 偏航、是否維持 Phase 0 / Phase 1 定案
acceptance criteria	所有 downstream changes 都能從 design.md 找到依據

### tasks.md

項目	內容
purpose	定義 change-01 的文件任務
required content	checkbox tasks
reviewer should check	tasks 是否全為文件任務，未進入實作
acceptance criteria	所有文件任務可逐項驗收

## specs/project-charter.md

項目	內容
purpose	存放正式 project charter
required content	mission、background、problem statement、proposed direction、success definition
reviewer should check	是否足以成為 source of truth
acceptance criteria	能支撐 README / proposal / downstream changes

## specs/scope.md

項目	內容
purpose	凍結 scope / non-scope
required content	in scope、out of scope、strict non-goals、expansion rules
reviewer should check	是否清楚禁止 IPFS / QUIC / multi-agent 等擴題
acceptance criteria	Antigravity 無法模糊解讀專案邊界

## specs/risk-register.md

項目	內容
purpose	管理專案風險與降階策略
required content	risk table、fallback principles、trigger conditions
reviewer should check	是否包含 ns3-gym、DQN、reward、baseline、scope drift 等核心風險
acceptance criteria	每個高風險都有 fallback

## 10. proposal.md 草案內容

### Markdown

#### # Change 01: Project Charter

#### ## Why

This project requires a source-of-truth document before any implementation begins. The final project has been scoped as a Deep Reinforcement Learning study for congestion control and throughput optimization over a single bottleneck link.

Without a project charter, the implementation agent may prematurely expand the project into IPFS, QUIC, kernel-level TCP, multi-agent reinforcement learning, or large-scale network simulation. This change establishes the project title, mission, research scope, non-goals, baseline plan, metrics, MVP definition, and downstream OpenSpec change structure.

#### ## What Changes

This change introduces the project charter for:

- Project title and identity
- Research mission
- Problem statement
- Proposed technical direction
- In-scope items
- Out-of-scope items
- Strict non-goals
- Research questions and hypotheses
- Baseline and metric freeze
- MVP success definition
- Downstream change dependency map
- Risk register

## ## What Does Not Change

This change does not implement any code.

This change does not:

- Create ns-3 experiments
- Create ns3-gym environments
- Train DQN or PPO agents
- Implement IPFS
- Implement QUIC
- Modify kernel-level TCP
- Add multi-agent reinforcement learning
- Add multi-path routing
- Run experiments
- Generate experimental results

## ## Impact

This charter becomes the upstream governance document for:

- `02-ns3-baseline-benchmark`
- `03-opengym-env`
- `04-drl-mvp`

All downstream changes must follow the scope, non-goals, MVP definition, and success criteria established here.

## ## Dependencies

This change depends on the Phase 0 and Phase 1 decisions:

- Final topic: Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link
- Toolchain direction: ns-3, ns3-gym, Stable-Baselines3
- MVP algorithm: DQN
- Future extension: PPO
- Baselines: NewReno, CUBIC, BBR
- Metrics: throughput, RTT, packet loss, utility, reward curve, convergence behavior

## ## Acceptance Criteria

- [ ] Project title is frozen.
- [ ] Repository name is defined.
- [ ] Project mission is defined.
- [ ] In-scope and out-of-scope items are clearly separated.
- [ ] Strict non-goals are explicit and enforceable.
- [ ] Baselines are frozen.
- [ ] Metrics are frozen.
- [ ] MVP success definition is clear.
- [ ] Downstream changes are mapped.
- [ ] Risk register is included.
- [ ] No implementation code is included.
- [ ] No shell commands are included.
- [ ] No experimental results are claimed.

# 11. design.md 草案内容

## Markdown

### # Design: Project Charter Governance

#### ## Design Goal

The goal of this design is to establish a governance-level project charter for a DRL final project on congestion control over a single bottleneck link.

The charter must ensure that the spec owner, Antigravity, OpenSpec artifacts, GitHub repository, presentation slides, and final video all follow the same project direction.

#### ## Project Scope Boundary

The project is scoped as a networking systems and deep reinforcement learning project.

The project focuses on:

- Single bottleneck link congestion control
- ns-3 simulation
- ns3-gym reinforcement learning interface
- Stable-Baselines3 DQN MVP
- Baseline comparison with NewReno, CUBIC, and preferably BBR
- Evaluation using throughput, RTT, packet loss, and utility

The project does not include:

- IPFS implementation
- QUIC implementation
- Kernel-level TCP modification
- Real Internet deployment
- Multi-agent reinforcement learning
- Multi-path routing
- Large-scale topology
- Production-grade congestion control protocol

#### ## Technical Direction

The technical direction is frozen as follows:

- Simulator: ns-3
- RL interface: ns3-gym
- Training framework: Stable-Baselines3
- MVP algorithm: DQN
- Future algorithm: PPO
- Baselines: NewReno, CUBIC, BBR
- Metrics: throughput, RTT, packet loss, utility, reward curve, convergence behavior

#### ## Governance Rules

1. No implementation may begin before this project charter is approved.
2. Antigravity must not write code during this change.
3. Antigravity must not create ns-3 experiments during this change.
4. Antigravity must not create DQN or PPO scripts during this change.
5. Antigravity must not expand the project into IPFS, QUIC, multi-agent RL, or multi-path routing.
6. Every downstream change must reference this charter.
7. If a downstream task conflicts with this charter, the task must stop and return to the spec owner.

#### ## Downstream Change Dependency

##### ### 02-ns3-baseline-benchmark

Depends on:

- Baseline list
- Metrics
- Single bottleneck scope
- Non-goals

##### ### 03-opengym-env

Depends on:

- MDP direction
- DQN discrete action requirement
- State / action / reward boundary
- ns3-gym decision

### ### 04-drl-mvp

Depends on:

- DQN as MVP algorithm
- Baseline comparison plan
- Metrics
- Evaluation philosophy

## ## Decision Records

### ### DR-01: Use single bottleneck link as the first scenario

Reason: It minimizes topology complexity while preserving the core congestion-control problem.

### ### DR-02: Use ns-3 as simulator

Reason: ns-3 supports TCP congestion-control baseline experiments and is suitable for research simulation.

### ### DR-03: Use ns3-gym as RL interface

Reason: It bridges ns-3 simulation and reinforcement learning workflows.

### ### DR-04: Use DQN for MVP

Reason: The MVP action space is discrete, and DQN is suitable for discrete action tasks.

### ### DR-05: Keep PPO as future extension

Reason: PPO is more flexible but should be introduced only after the DQN MVP and baseline benchmark are stable.

### ### DR-06: Do not implement IPFS in this final project

Reason: IPFS is used as long-term motivation and knowledge continuity, not as the implementation target for this course project.

## 12. tasks.md 草案内容

### Markdown

## # Tasks: Change 01 Project Charter

### ## Documentation Tasks

- [ ] Confirm final Chinese project title.
- [ ] Confirm final English project title.
- [ ] Confirm GitHub repository name.
- [ ] Confirm project mission.
- [ ] Confirm project background.
- [ ] Confirm problem statement.
- [ ] Confirm proposed direction.
- [ ] Confirm success definition.
- [ ] Confirm in-scope items.
- [ ] Confirm out-of-scope items.
- [ ] Confirm strict non-goals.
- [ ] Confirm main research question.
- [ ] Confirm sub-questions.
- [ ] Confirm hypotheses.
- [ ] Confirm simulator decision: ns-3.
- [ ] Confirm RL interface decision: ns3-gym.
- [ ] Confirm training framework decision: Stable-Baselines3.
- [ ] Confirm MVP algorithm decision: DQN.
- [ ] Confirm future algorithm decision: PPO.
- [ ] Confirm baseline list: NewReno, CUBIC, BBR.
- [ ] Confirm metrics: throughput, RTT, packet loss, utility, reward curve, convergence behavior.

- [ ] Confirm MVP must-have items.
- [ ] Confirm MVP should-have items.
- [ ] Confirm MVP could-have items.
- [ ] Confirm MVP must-not-have items.
- [ ] Confirm MVP acceptance criteria.
- [ ] Confirm downstream change map.
- [ ] Confirm risk register.
- [ ] Confirm OpenSpec artifact blueprint.
- [ ] Review all documents for accidental implementation details.
- [ ] Review all documents for scope drift.
- [ ] Submit change 01 to spec owner for review.

## 13. specs 草案摘要

### 13.1 specs/project-charter.md

#### Purpose

建立本專題的上游 source of truth，確保所有 proposal、GitHub、PPT、影片與 Antigravity implementation 都對齊同一題目與範圍。

#### Project mission

將單一瓶頸鏈路壅塞控制建模為 DRL sequential decision-making problem，規劃可重現的 ns-3 / ns3-gym / SB3 workflow，並與傳統 TCP baseline 比較 throughput、RTT、loss 與 utility。

#### Research target

- single bottleneck congestion control
- sender-side control abstraction
- DQN MVP
- baseline comparison with NewReno / CUBIC / BBR

#### Success definition

成功是建立可執行規格與後續 implementation roadmap，不是保證 DRL 贏過所有 baseline。

#### OpenSpec governance role

OpenSpec 是所有 change 的規格中樞。後續實作必須由已驗收 change 驅動，不得直接由聊天內容或 Antigravity 自由發揮。

### 13.2 specs/scope.md

#### In scope

- single bottleneck link
- ns-3 simulation
- ns3-gym RL interface

- DQN MVP
- NewReno / CUBIC / BBR baseline
- throughput / RTT / loss / utility evaluation
- OpenSpec-driven workflow

## Out of scope

- IPFS implementation
- QUIC implementation
- kernel-level TCP
- real Internet deployment
- multi-agent RL
- multi-path routing
- large-scale topology
- production-level protocol

## Strict non-goals

- 不得跳過 baseline 直接做 DRL。
- 不得只最佳化 throughput。
- 不得把 IPFS 寫成主體。
- 不得把 PPO 當 MVP 必做。
- 不得把 Pantheon 當必裝依賴。

## Expansion rules

任何擴展只能在 MVP 完成後，由 spec owner 另開 change 審核。

## 13.3 specs/risk-register.md

### Risk categories

- Toolchain risk
- Baseline risk
- RL training risk
- Reward design risk
- Scope drift risk
- Reporting risk
- Governance risk

### Risk table summary

至少包含 R1–R10，包括 ns3-gym 安裝、baseline logging、BBR 版本、DQN 不收斂、reward 學歪、Antigravity 擴題、題目被誤解成 IPFS、結果未超越 baseline、圖表不足、OpenSpec 與實作脫節。

## Fallback principles

- baseline 先行
- DQN 失敗時保留 trade-off analysis
- BBR 失敗時降階為 NewReno + CUBIC
- ns3-gym 失敗時啟動 simplified Gym fallback
- scope drift 發生時停止實作並回到 spec review

## 14. Review Checklist

### change-01 驗收檢查表

- ☐ 題目是否凍結？
- ☐ 英文題目是否凍結？
- ☐ repo name 是否凍結？
- ☐ project mission 是否清楚？
- ☐ problem statement 是否清楚？
- ☐ proposed direction 是否清楚？
- ☐ success definition 是否不過度承諾？
- ☐ scope 是否清楚？
- ☐ non-scope 是否清楚？
- ☐ strict non-goals 是否足夠強硬？
- ☐ baseline 是否凍結？
- ☐ metrics 是否凍結？
- ☐ MVP 是否凍結？
- ☐ DQN 是否被定義為 MVP？
- ☐ PPO 是否被定義為 future extension？
- ☐ IPFS 是否只作為背景動機，而不是實作主體？
- ☐ QUIC 是否被排除？
- ☐ kernel-level TCP 是否被排除？
- ☐ multi-agent / multi-path 是否被排除？
- ☐ change-02 是否能接上？
- ☐ change-03 是否能接上？
- ☐ change-04 是否能接上？

- ☐ risk register 是否包含必要風險？
- ☐ 每個高風險是否都有 fallback？
- ☐ 是否有任何內容偷偷進入實作？
- ☐ 是否有任何 shell command？
- ☐ 是否有任何 code？
- ☐ 是否有任何內容偏離 Phase 0 / Phase 1 定案？
- ☐ 是否可作為 Antigravity 的上游指令依據？

## 15. 給 Antigravity 的 change-01 指令草案

You are working on Phase 2 / OpenSpec change 01: project-charter.

Current task:

Create only the OpenSpec project-charter artifacts for the DRL final project.

Project:

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

Repository:

drl-bottleneck-congestion-control

You must only produce documentation artifacts for:

- openspec/changes/01-project-charter/proposal.md
- openspec/changes/01-project-charter/design.md
- openspec/changes/01-project-charter/tasks.md
- openspec/changes/01-project-charter/specs/project-charter.md
- openspec/changes/01-project-charter/specs/scope.md
- openspec/changes/01-project-charter/specs/risk-register.md

Strict rules:

- Do not write implementation code.
- Do not create ns-3 experiments.
- Do not create ns3-gym environment code.
- Do not create DQN or PPO training scripts.
- Do not run baseline benchmarks.
- Do not implement IPFS.
- Do not implement QUIC.
- Do not modify Linux kernel TCP.
- Do not introduce multi-agent RL.
- Do not introduce multi-path routing.
- Do not change the project topic.
- Do not optimize for throughput alone.
- Do not claim experimental results.

The purpose of this change is to create the source-of-truth project charter. The charter must freeze:

- project title
- mission
- scope
- non-goals
- strict non-goals
- technical direction
- MVP definition
- baselines
- metrics
- downstream change map
- risk register

After completing the documentation artifacts, stop and wait for spec owner review.

## change-01 結論

**OpenSpec change 01 : project-charter** 應該被視為本專題真正的第一份治理文件，而不是一般摘要。它的驗收重點不是「有沒有開始做實驗」，而是「是否已經把後續所有實作可走與不可走的路全部鎖清楚」。本 change 完成並驗收後，才能進入 **change-02-baseline-benchmark**；在那之前，不應啟動 ns-3 baseline、不應串 ns3-gym、不應訓練 DQN。

OpenSpec change 01 : project-charter 階段驗收沒問題

請用一樣的生成標準（最高）產出「change-02-baseline-benchmark」的最佳 prompt

收到。以下是 **Phase 2 / OpenSpec change 02 : baseline-benchmark** 的最高品質最佳 prompt。

這一階段的重點不是跑 ns-3，也不是開始產生 baseline 圖，而是先把 **baseline benchmark 的規格真相來源** 寫清楚，讓後續 Phase 3「Baseline 先行」可以照規格執行。ns-3 官方 TCP model 支援多種 TCP congestion control 演算法，包含 NewReno、CUBIC、BBR 等；Flow Monitor 官方文件也說明它可用於量測網路協定效能；ns3-gym 則是把 ns-3 與 RL 研究介面整合的框架。這些會被要求寫入 change-02 的 benchmark 規格基礎。 NSNAM+2

同時，OpenSpec 的工作方式是以 change folder 管理 proposal、specs、design、tasks，並在 implementation 前先完成規格對齊；所以這份 prompt 會強制產出 **baseline benchmark 的 proposal.md / design.md / tasks.md / specs**，但不允許執行任何程式。 GitHub+1

請下一回合直接貼上以下整段：

你現在要執行的是我 DRL Final Project 的 **\*\*Phase 2：用 OpenSpec 建立專案真相來源\*\*** 的第二階段：

# OpenSpec change 02:baseline-benchmark

## 最高任務目標

你要基於已驗收完成的：

- change-01-project-charter
- Phase 0 定案
- Phase 1 proposal 主體

替我產出一份可以作為後續 **\*\*Phase 3:Baseline 先行\*\*** 的 OpenSpec-style baseline benchmark 規格文件。

這不是實作任務，不是 ns-3 coding，不是跑 baseline，不是產圖。

這是 baseline benchmark 的 **\*\*source of truth\*\***。

本 change 的唯一目標是把後續 baseline 實作要做什麼、怎麼量測、怎麼驗收、哪些是必做、哪些是加分、哪些不能碰，全部寫成可交給 Antigravity 執行的規格。

---

# 一、已驗收前置內容

## 1. 已完成 change

`01-project-charter` 已驗收通過。

該 change 已凍結：

- 題目
- scope
- non-goals
- baseline list
- metrics
- MVP definition
- risk register
- downstream change map

本 change 必須完全繼承，不得自行改題、擴題或跳到 DRL。

---

## ## 2. 正式題目

### ### 中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

### ### 英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

### ### GitHub repo name

drl-bottleneck-congestion-control

---

## ## 3. 本 change 的定位

本 change 是：

- baseline benchmark 規格階段
- ns-3 baseline 實驗設計階段
- metrics / logging / scenario / acceptance criteria 凍結階段
- change-03-opengym-env 與 change-04-drl-mvp 的前置依據

本 change 不是：

- ns-3 實作階段
- Python 訓練階段
- ns3-gym environment 建立階段
- DQN 訓練階段
- PPO 擴充階段
- IPFS / QUIC / kernel TCP 實作階段

---

## # 二、change-02 的核心目的

change-02-baseline-benchmark 的任務是建立 **傳統 TCP baseline benchmark 的規格**。

它要回答：

1. baseline benchmark 要測什麼？
2. 為什麼 baseline 必須先於 DRL？
3. single bottleneck link 場景要如何定義？
4. NewReno / CUBIC / BBR 各自扮演什麼 baseline 角色？
5. throughput / RTT / loss / utility 要怎麼定義？
6. baseline 實驗至少要有哪些 scenario？
7. baseline 結果要輸出哪些 artifacts？
8. 後續 change-03 / change-04 如何使用這些 baseline？
9. 若 BBR 或 logging 有風險，怎麼降階？
10. Antigravity 後續 Phase 3 執行時，如何避免偏離 benchmark 規格？

---

### # 三、本階段嚴格禁止事項

本 change 嚴格禁止：

- 不寫任何 C++ code
- 不寫任何 Python code
- 不寫 shell command
- 不建立 ns-3 topology 實作
- 不執行 ns-3
- 不產生實驗數據
- 不產生圖表
- 不啟動 ns3-gym
- 不定義 RL agent implementation
- 不訓練 DQN
- 不接 PPO
- 不接 IPFS
- 不接 QUIC
- 不修改 Linux kernel TCP
- 不做 multi-agent
- 不做 multi-path routing
- 不做大型 topology
- 不把 Pantheon 設為必裝依賴
- 不宣稱已有 baseline 實驗結果

你只能產出 OpenSpec 文件草案與規格內容。

---

### # 四、你必須先搜尋並引用的來源

請在正式輸出前先做最新網路搜尋，不可只靠記憶。

#### ## 優先使用來源

請優先搜尋並引用：

1. OpenSpec 官方網站 / GitHub README / docs
2. ns-3 官方 TCP model 文件
3. ns-3 Flow Monitor 官方文件
4. ns-3 tracing / statistics / measurement 相關官方文件，如有需要
5. ns3-gym 官方 GitHub / ns-3 app 頁面
6. Aurora 原始論文
7. Pantheon paper / 官方 benchmark 說明
8. 必要時補充正式論文或 RFC

#### ## 你必須用來源確認的判斷

請確認並支撐以下判斷：

- OpenSpec 適合用來建立 change-based benchmark 規格
- ns-3 適合做 TCP / congestion control baseline
- ns-3 支援 NewReno、CUBIC、BBR 等 TCP congestion control
- Flow Monitor 或 ns-3 內建工具可支援 throughput / delay / loss 類指標量測
- baseline-first 是合理順序，因為後續 DRL 必須有比較基準
- Pantheon 適合作為 benchmark / reproducibility 思想來源，但不是 MVP 必裝依賴
- throughput 不能是唯一 baseline metric，必須同時規格化 RTT / loss / utility
- baseline benchmark 的結果將作為 change-04-drl-mvp 的比較基準

---

### # 五、change-02 必須產出的內容

請嚴格按照以下章節輸出。

---

## ## # OpenSpec change 02:baseline-benchmark 總結

請用 150–250 字說明：

- change 02 的目的
- 它如何承接 change 01
- 它為什麼必須在 DRL 前完成
- 它如何約束 change 03 / change 04
- 它如何避免 Antigravity 直接跳到 DQN

---

## ## # 1. Change Metadata

請輸出：

- Change ID
- Change title
- Change type
- Status
- Owner
- Implementer
- Related phase
- Depends on
- Related downstream changes
- Required before implementation:Yes / No
- Code implementation allowed in this change:Yes / No

請明確設定：

- Change ID:02-baseline-benchmark
- Change type:benchmark specification / experiment planning / metrics definition
- Status:draft for review
- Owner:user / spec owner
- Implementer:Antigravity, after approval
- Depends on:01-project-charter
- Related downstream changes:03-opengym-env, 04-drl-mvp
- Code implementation allowed:No

---

## ## # 2. Baseline Benchmark Charter

請產出正式 baseline benchmark charter。

### ### 2.1 Benchmark Mission

請說明：

- baseline benchmark 的使命
- 為什麼 baseline 必須先行
- 為什麼不能直接跳到 DRL
- baseline 對後續 DQN 評估的作用

### ### 2.2 Benchmark Scope

請列出 baseline benchmark 要涵蓋：

- single bottleneck link
- sender / receiver
- TCP traffic flow
- NewReno
- CUBIC

- BBR
- throughput / RTT / loss / utility
- planned scenarios
- logs / tables / figures 的規格

### ### 2.3 Benchmark Non-Scope

請列出 baseline benchmark 不做：

- 不做 DRL
- 不做 ns3-gym
- 不做 DQN / PP0
- 不做 IPFS
- 不做 QUIC
- 不做 kernel TCP
- 不做 multi-agent
- 不做 multi-path
- 不做 large-scale topology
- 不做 Pantheon full integration

### ### 2.4 Benchmark Success Definition

成功不等於：

- baseline 已經跑出最好結果
- baseline 圖已經產生
- DRL 已經比較完成

成功等於：

- baseline methods 明確
- topology requirements 明確
- logging requirements 明確
- metrics definitions 明確
- scenario matrix 明確
- fallback 明確
- downstream changes 可依此使用 benchmark 規格

---

## ## # 3. Baseline Methods Specification

請正式定義 baseline methods。

### ### 3.1 NewReno

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 它和 DQN 的比較意義

### ### 3.2 CUBIC

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 若 ns-3 預設為 CUBIC，這對 benchmark 有什麼意義

### ### 3.3 BBR

請說明：

- 為什麼選它

- 它在本專題中的 baseline 角色
- 為什麼它是 strongly recommended 而不是 hard MVP requirement
- 如果 BBR 有版本相依性或跑不起來，如何降階

### ### 3.4 Optional Baselines

請列出加分但不進 MVP hard requirement 的 baseline：

- Vegas
- Westwood
- DCTCP
- LEDBAT

請說明為什麼不把它們列入必做。

---

## ## # 4. Topology Specification

請定義 single bottleneck link 的 benchmark topology。

### ### 4.1 Topology Concept

請用文字描述：

- sender
- receiver
- bottleneck link
- access links
- queue / buffer
- optional cross traffic

### ### 4.2 Minimal Topology

請定義最小拓模：

- one sender
- one receiver
- one bottleneck link
- one long-lived TCP flow

### ### 4.3 Enhanced Topology

請定義加分拓模：

- cross traffic
- variable bandwidth
- high-delay path
- multiple flows, 但只能作為 future extension, 不是 MVP

### ### 4.4 Topology Diagram Description

請用文字描述一張後續應畫出的圖：

Sender → Access Link → Bottleneck Queue / Link → Access Link → Receiver

請不要畫圖，只描述圖的構成與標籤。

---

## ## # 5. Metrics and Logging Specification

這是本 change 最重要的部分之一。

請定義所有 baseline benchmark metrics。

### ### 5.1 Average Throughput

請定義：

- metric meaning

- why it matters
- expected unit
- data needed
- possible calculation concept
- logging requirement

### ### 5.2 Average RTT / Delay

請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- logging requirement
- 和 bufferbloat / queueing delay 的關係

### ### 5.3 Packet Loss Rate

請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- logging requirement

### ### 5.4 Utility Score

請定義：

- 為什麼需要 utility
- 為什麼不能只看 throughput
- 建議的概念形式

例如：

$utility = normalized\_throughput - penalty\_delay - penalty\_loss$

請提供：

- 符號注釋
- 白話說明
- 注意此階段不固定最終權重，只固定 utility philosophy

### ### 5.5 Output Artifacts

請定義後續 Phase 3 至少要輸出的 artifacts：

- raw logs
- summary CSV
- per-scenario summary table
- baseline comparison table
- throughput plot
- RTT plot
- packet loss plot
- utility comparison plot
- experiment metadata

注意：

這裡只定義 artifacts，不產生 artifacts。

---

## ## # 6. Scenario Matrix Specification

請設計 baseline benchmark scenario matrix。

至少包含 4 組情境：

1. Stable low-delay bottleneck
2. Stable high-delay bottleneck
3. Variable bandwidth bottleneck
4. Bottleneck with cross traffic

每組情境請提供：

- Scenario ID
- Purpose
- Bandwidth profile
- Delay profile
- Queue / buffer setting
- Traffic pattern
- Required baselines
- Required metrics
- Expected observation
- Risk level
- Whether MVP-required

請注意：

- 不要填過度細的實作參數
- 可以用 reasonable placeholder
- 但要足夠讓 Antigravity 後續實作時不偏航

---

## ## # 7. Benchmark Evaluation Philosophy

請明確定義本專題 baseline benchmark 的評估哲學。

必須包含：

- baseline 不是敵人，而是比較基準
- 不以單一 throughput 最大化為唯一成功標準
- delay / RTT / loss 是同等重要的分析維度
- DRL 後續若輸給 BBR / CUBIC 也不是失敗，只要 trade-off analysis 完整
- baseline 的作用是提供後續 DQN 的 reference point
- baseline 實驗應可重現、可比較、可視覺化
- Pantheon 是 benchmark 思想來源，但不是本 change 的必裝工具

---

## ## # 8. Downstream Dependency

請定義 change-02 如何餵給後續 change。

### ### 8.1 對 change-03-opengym-env 的輸入

請說明 baseline benchmark 如何幫助定義：

- observation candidates
- metric logging
- reward components
- scenario reuse
- episode / evaluation setup

### ### 8.2 對 change-04-drl-mvp 的輸入

請說明 baseline benchmark 如何幫助定義：

- DQN comparison target
- evaluation metrics
- reward curve interpretation
- success / failure analysis
- final project figures

---

## ## # 9. Risk Register for Baseline Benchmark

請建立 change-02 專屬風險清單。

至少包含：

1. ns-3 TCP algorithm name / version mismatch
2. BBR availability issue
3. FlowMonitor / logging 不足以取得 RTT 或 loss
4. throughput 計算口徑不一致
5. scenario 設計太複雜
6. cross traffic 讓 MVP 失控
7. baseline 結果異常但無法解釋
8. Antigravity 直接跳到 DRL
9. baseline 圖表不足以支撐 PPT
10. Pantheon 被誤當必裝工具

表格欄位請包含：

- Risk ID
- Description
- Probability
- Impact
- Prevention
- Fallback
- Owner
- Trigger condition

---

## # 10. OpenSpec Artifact Blueprint

請設計 change-02-baseline-benchmark 應產出的 artifact。

請不要真的建立檔案，但要提供建議結構與內容摘要。

至少包含：

```
```text
openspec/
  changes/
    02-baseline-benchmark/
      proposal.md
      design.md
      tasks.md
  specs/
    baseline-methods.md
    topology.md
    metrics-logging.md
    scenario-matrix.md
    benchmark-risk-register.md
```

對每個檔案說明：

- purpose
- required content
- reviewer should check
- acceptance criteria

11. proposal.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/02-baseline-benchmark/proposal.md
```

的 markdown 草案。

內容至少包含：

- Why
- What Changes
- What Does Not Change
- Impact
- Dependencies
- Acceptance Criteria

注意：

這是 OpenSpec change proposal，不是論文 proposal。

12. design.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/02-baseline-benchmark/design.md
```

的 markdown 草案。

內容至少包含：

- Design Goal
- Benchmark Boundary
- Baseline Methods
- Topology Direction
- Metrics and Logging Direction
- Scenario Direction
- Downstream Usage
- Decision Records

注意：

design.md 只處理 benchmark 設計，不處理實作程式。

13. tasks.md 草案內容

請直接產出一份可以放入：

```
openspec/changes/02-baseline-benchmark/tasks.md
```

的 markdown 草案。

tasks 必須是規格任務，不是實作任務。

例如：

- Confirm baseline methods
- Confirm topology spec
- Confirm metrics definitions
- Confirm scenario matrix
- Confirm output artifacts
- Review no implementation included

每個 task 請用 checkbox 格式。

14. specs 草案摘要

請產出 specs 下五個檔案的摘要內容，不必寫成完整檔案，但要夠後續轉成完整 spec。

14.1 specs/baseline-methods.md

請提供：

- Purpose
- Required baselines
- Strongly recommended baselines
- Optional baselines
- Exclusion rules

14.2 specs/topology.md

請提供：

- Purpose
- Minimal topology
- Enhanced topology
- Topology assumptions
- Non-goals

14.3 specs/metrics-logging.md

請提供：

- Purpose

- Metric definitions
- Required logs
- Required output artifacts
- Ambiguity rules

14.4 specs/scenario-matrix.md

請提供：

- Purpose
- Required scenarios
- Scenario table summary
- MVP-required vs optional

14.5 specs/benchmark-risk-register.md

請提供：

- Purpose
 - Risk categories
 - Risk summary
 - Fallback principles
-

15. Review Checklist

請建立 change-02 驗收檢查表。

檢查表必須讓我能回答：

- baseline methods 是否明確？
- NewReno / CUBIC / BBR 角色是否清楚？
- BBR 是否有 fallback？
- topology 是否清楚？
- metrics 是否清楚？
- logging artifacts 是否清楚？
- scenario matrix 是否清楚？
- 是否避免直接進入 DRL？
- 是否避免實作？
- 是否能餵給 change-03 / change-04？
- 是否符合 change-01 的 scope / non-goals？

請用 checklist 格式輸出。

16. 給 Antigravity 的 change-02 指令草案

請幫我產出一段可以交給 Antigravity 的 change-02 指令。

這段指令必須說清楚：

- 目前只做 baseline benchmark specification
- 不准寫 code
- 不准跑 ns-3
- 不准啟動 ns3-gym
- 不准開始 DQN
- 不准接 PPO
- 不准加入 IPFS / QUIC / multi-agent / multi-path
- 必須只產出 OpenSpec artifacts
- 必須遵守 change-01 project-charter
- 完成後等待 spec owner 驗收

六、輸出品質要求

1. 語言

- 全文使用繁體中文
- 必要英文專有名詞保留英文
- 檔名、change ID、artifact 名稱保持英文

2. 深度

不要只列大綱。

要寫到我可以直接複製部分內容去建立 OpenSpec 文件。

3. 嚴謹度

所有重要判斷要有可信來源支撐。

若是推論，請明確寫「這是基於來源的推論」。

4. 禁止事項

本階段禁止：

- 寫任何程式碼
- 寫 shell command
- 產生 ns-3 實作
- 產生 Python 訓練腳本
- 執行 baseline benchmark
- 產出實驗圖表
- 進入 opengym-env
- 進入 drl-mvp
- 更改題目
- 擴張到 IPFS / QUIC / multi-agent / multi-path
- 宣稱已有 baseline 實驗結果

七、現在開始執行

請先搜尋官方文件與正式來源，再輸出。

請直接從：

OpenSpec change 02：baseline-benchmark 總結

開始。

收到。以下是 Phase 2 / OpenSpec change 02：baseline-benchmark 的最高品質最佳 prompt。

這一階段的重點不是跑 ns-3，也不是開始產生 baseline 圖，而是先把 baseline benchmark 的規格真相來源寫清楚，讓後續 Phase 3「Baseline 先行」可以照規格執行。ns-3 官方 TCP model 支援多種 TCP congestion control 演算法，包含 NewReno、CUBIC、BBR 等；Flow Monitor 官方文件也說明它可用於量測網路協定效能；ns3-gym 則是把 ns-3 與 RL 研究介面整合的框架。這些會被要求寫入 change-02 的 benchmark 規格基礎。

同時，OpenSpec 的工作方式是以 change folder 管理 proposal、specs、design、tasks，並在 implementation 前先完成規格對齊；所以這份 prompt 會強制產出 baseline benchmark 的 proposal.md / design.md / tasks.md / specs，但不允許執行任何程式。

請下一回合直接貼上以下整段：

你現在要執行的是我 DRL Final Project 的 **Phase 2：用 OpenSpec 建立專案真相來源** 的第二階段：

OpenSpec change 02：baseline-benchmark

最高任務目標

你要基於已驗收完成的：

- change-01-project-charter
- Phase 0 定案
- Phase 1 proposal 主體

替我產出一份可以作為後續 **Phase 3：Baseline 先行** 的 OpenSpec-style baseline benchmark 規格文件。

這不是實作任務，不是 ns-3 coding，不是跑 baseline，不是產圖。
這是 baseline benchmark 的 **source of truth**。

本 change 的唯一目標是把後續 baseline 實作要做什麼、怎麼量測、怎麼驗收、哪些是必做、哪些是加分、哪些不能碰，全部寫成可交給 Antigravity 執行的規格。

一、已驗收前置內容

1. 已完成 change

01-project-charter 已驗收通過。

該 change 已凍結：

- 題目
- scope
- non-goals
- baseline list
- metrics
- MVP definition
- risk register
- downstream change map

本 change 必須完全繼承，不得自行改題、擴題或跳到 DRL。

2. 正式題目

中文題目

以深度強化學習進行單一瓶頸鏈路壅塞控制與吞吐量最佳化

英文題目

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

GitHub repo name

drl-bottleneck-congestion-control

3. 本 change 的定位

本 change 是：

- baseline benchmark 規格階段
- ns-3 baseline 實驗設計階段
- metrics / logging / scenario / acceptance criteria 凍結階段
- change-03-opengym-env 與 change-04-drl-mvp 的前置依據

本 change 不是：

- ns-3 實作階段
- Python 訓練階段
- ns3-gym environment 建立階段
- DQN 訓練階段
- PPO 擴充階段
- IPFS / QUIC / kernel TCP 實作階段

二、change-02 的核心目的

change-02-baseline-benchmark 的任務是建立 **傳統 TCP baseline benchmark 的規格**。

它要回答：

1. baseline benchmark 要測什麼？
2. 為什麼 baseline 必須先於 DRL？
3. single bottleneck link 場景要如何定義？
4. NewReno / CUBIC / BBR 各自扮演什麼 baseline 角色？
5. throughput / RTT / loss / utility 要怎麼定義？
6. baseline 實驗至少要有哪些 scenario？
7. baseline 結果要輸出哪些 artifacts？
8. 後續 change-03 / change-04 如何使用這些 baseline？
9. 若 BBR 或 logging 有風險，怎麼降階？
10. Antigravity 後續 Phase 3 執行時，如何避免偏離 benchmark 規格？

三、本階段嚴格禁止事項

本 change 嚴格禁止：

- 不寫任何 C++ code
- 不寫任何 Python code

- 不寫 shell command
- 不建立 ns-3 topology 實作
- 不執行 ns-3
- 不產生實驗數據
- 不產生圖表
- 不啟動 ns3-gym
- 不定義 RL agent implementation
- 不訓練 DQN
- 不接 PPO
- 不接 IPFS
- 不接 QUIC
- 不修改 Linux kernel TCP
- 不做 multi-agent
- 不做 multi-path routing
- 不做大型 topology
- 不把 Pantheon 設為必裝依賴
- 不宣稱已有 baseline 實驗結果

你只能產出 OpenSpec 文件草案與規格內容。

四、你必須先搜尋並引用的來源

請在正式輸出前先做最新網路搜尋，不可只靠記憶。

優先使用來源

請優先搜尋並引用：

1. OpenSpec 官方網站 / GitHub README / docs
2. ns-3 官方 TCP model 文件
3. ns-3 Flow Monitor 官方文件
4. ns-3 tracing / statistics / measurement 相關官方文件，如有需要
5. ns3-gym 官方 GitHub / ns-3 app 頁面
6. Aurora 原始論文
7. Pantheon paper / 官方 benchmark 說明
8. 必要時補充正式論文或 RFC

你必須用來源確認的判斷

請確認並支撐以下判斷：

- OpenSpec 適合用來建立 change-based benchmark 規格
- ns-3 適合做 TCP / congestion control baseline
- ns-3 支援 NewReno、CUBIC、BBR 等 TCP congestion control
- Flow Monitor 或 ns-3 內建工具可支援 throughput / delay / loss 類指標量測

- baseline-first 是合理順序，因為後續 DRL 必須有比較基準
- Pantheon 適合作為 benchmark / reproducibility 思想來源，但不是 MVP 必裝依賴
- throughput 不能是唯一 baseline metric，必須同時規格化 RTT / loss / utility
- baseline benchmark 的結果將作為 change-04-drl-mvp 的比較基準

五、change-02 必須產出的內容

請嚴格按照以下章節輸出。

OpenSpec change 02：baseline-benchmark 總結

請用 150-250 字說明：

- change 02 的目的
- 它如何承接 change 01
- 它為什麼必須在 DRL 前完成
- 它如何約束 change 03 / change 04
- 它如何避免 Antigravity 直接跳到 DQN

1. Change Metadata

請輸出：

- Change ID
- Change title
- Change type
- Status
- Owner
- Implementer
- Related phase
- Depends on
- Related downstream changes
- Required before implementation：Yes / No
- Code implementation allowed in this change：Yes / No

請明確設定：

- Change ID：02-baseline-benchmark
- Change type：benchmark specification / experiment planning / metrics definition
- Status：draft for review
- Owner：user / spec owner
- Implementer：Antigravity, after approval
- Depends on：01-project-charter

- Related downstream changes : 03-opengym-env, 04-drl-mvp
- Code implementation allowed : No

2. Baseline Benchmark Charter

請產出正式 baseline benchmark charter。

2.1 Benchmark Mission

請說明：

- baseline benchmark 的使命
- 為什麼 baseline 必須先行
- 為什麼不能直接跳到 DRL
- baseline 對後續 DQN 評估的作用

2.2 Benchmark Scope

請列出 baseline benchmark 要涵蓋：

- single bottleneck link
- sender / receiver
- TCP traffic flow
- NewReno
- CUBIC
- BBR
- throughput / RTT / loss / utility
- planned scenarios
- logs / tables / figures 的規格

2.3 Benchmark Non-Scope

請列出 baseline benchmark 不做：

- 不做 DRL
- 不做 ns3-gym
- 不做 DQN / PPO
- 不做 IPFS
- 不做 QUIC
- 不做 kernel TCP
- 不做 multi-agent
- 不做 multi-path
- 不做 large-scale topology
- 不做 Pantheon full integration

2.4 Benchmark Success Definition

成功不等於：

- baseline 已經跑出最好結果
- baseline 圖已經產生
- DRL 已經比較完成

成功等於：

- baseline methods 明確
- topology requirements 明確
- logging requirements 明確
- metrics definitions 明確
- scenario matrix 明確
- fallback 明確
- downstream changes 可依此使用 benchmark 規格

3. Baseline Methods Specification

請正式定義 baseline methods。

3.1 NewReno

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 它和 DQN 的比較意義

3.2 CUBIC

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 若 ns-3 預設為 CUBIC，這對 benchmark 有什麼意義

3.3 BBR

請說明：

- 為什麼選它
- 它在本專題中的 baseline 角色
- 為什麼它是 strongly recommended 而不是 hard MVP requirement
- 如果 BBR 有版本相依性或跑不起來，如何降階

3.4 Optional Baselines

請列出加分但不進 MVP hard requirement 的 baseline：

- Vegas
- Westwood
- DCTCP
- LEDBAT

請說明為什麼不把它們列入必做。

4. Topology Specification

請定義 single bottleneck link 的 benchmark topology。

4.1 Topology Concept

請用文字描述：

- sender
- receiver
- bottleneck link
- access links
- queue / buffer
- optional cross traffic

4.2 Minimal Topology

請定義最小拓樸：

- one sender
- one receiver
- one bottleneck link
- one long-lived TCP flow

4.3 Enhanced Topology

請定義加分拓樸：

- cross traffic
- variable bandwidth
- high-delay path
- multiple flows, 但只能作為 future extension, 不是 MVP

4.4 Topology Diagram Description

請用文字描述一張後續應畫出的圖：

Sender → Access Link → Bottleneck Queue / Link → Access Link → Receiver

請不要畫圖，只描述圖的構成與標籤。

5. Metrics and Logging Specification

這是本 change 最重要的部分之一。

請定義所有 baseline benchmark metrics。

5.1 Average Throughput

請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- possible calculation concept
- logging requirement

5.2 Average RTT / Delay

請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- logging requirement
- 和 bufferbloat / queueing delay 的關係

5.3 Packet Loss Rate

請定義：

- metric meaning
- why it matters
- expected unit
- data needed
- logging requirement

5.4 Utility Score

請定義：

- 為什麼需要 utility
- 為什麼不能只看 throughput
- 建議的概念形式

例如：

utility = normalized_throughput - penalty_delay - penalty_loss

請提供：

- 符號注釋
- 白話說明
- 注意此階段不固定最終權重，只固定 utility philosophy

5.5 Output Artifacts

請定義後續 Phase 3 至少要輸出的 artifacts：

- raw logs
- summary CSV
- per-scenario summary table
- baseline comparison table
- throughput plot
- RTT plot
- packet loss plot
- utility comparison plot
- experiment metadata

注意：

這裡只定義 artifacts，不產生 artifacts。

6. Scenario Matrix Specification

請設計 baseline benchmark scenario matrix。

至少包含 4 組情境：

1. Stable low-delay bottleneck
2. Stable high-delay bottleneck
3. Variable bandwidth bottleneck
4. Bottleneck with cross traffic

每組情境請提供：

- Scenario ID
- Purpose
- Bandwidth profile
- Delay profile
- Queue / buffer setting
- Traffic pattern
- Required baselines

- Required metrics
- Expected observation
- Risk level
- Whether MVP-required

請注意：

- 不要填過度細的實作參數
- 可以用 reasonable placeholder
- 但要足夠讓 Antigravity 後續實作時不偏航

7. Benchmark Evaluation Philosophy

請明確定義本專題 baseline benchmark 的評估哲學。

必須包含：

- baseline 不是敵人，而是比較基準
- 不以單一 throughput 最大化為唯一成功標準
- delay / RTT / loss 是同等重要的分析維度
- DRL 後續若輸給 BBR / CUBIC 也不是失敗，只要 trade-off analysis 完整
- baseline 的作用是提供後續 DQN 的 reference point
- baseline 實驗應可重現、可比較、可視覺化
- Pantheon 是 benchmark 思想來源，但不是本 change 的必裝工具

8. Downstream Dependency

請定義 change-02 如何餵給後續 change。

8.1 對 change-03-opengym-env 的輸入

請說明 baseline benchmark 如何幫助定義：

- observation candidates
- metric logging
- reward components
- scenario reuse
- episode / evaluation setup

8.2 對 change-04-drl-mvp 的輸入

請說明 baseline benchmark 如何幫助定義：

- DQN comparison target
- evaluation metrics

- reward curve interpretation
- success / failure analysis
- final project figures

9. Risk Register for Baseline Benchmark

請建立 change-02 專屬風險清單。

至少包含：

1. ns-3 TCP algorithm name / version mismatch
2. BBR availability issue
3. FlowMonitor / logging 不足以取得 RTT 或 loss
4. throughput 計算口徑不一致
5. scenario 設計太複雜
6. cross traffic 讓 MVP 失控
7. baseline 結果異常但無法解釋
8. Antigravity 直接跳到 DRL
9. baseline 圖表不足以支撐 PPT
10. Pantheon 被誤當必裝工具

表格欄位請包含：

- Risk ID
- Description
- Probability
- Impact
- Prevention
- Fallback
- Owner
- Trigger condition

10. OpenSpec Artifact Blueprint

請設計 change-02-baseline-benchmark 應產出的 artifact。

請不要真的建立檔案，但要提供建議結構與內容摘要。

至少包含：

```
text
openspec/
  changes/
    02-baseline-benchmark/
      proposal.md
      design.md
      tasks.md
```

```
specs/
baseline-methods.md
topology.md
metrics-logging.md
scenario-matrix.md
benchmark-risk-register.md
```

對每個檔案說明：

- purpose
- required content
- reviewer should check
- acceptance criteria

11. proposal.md 草案內容

請直接產出一份可以放入：

`openspec/changes/02-baseline-benchmark/proposal.md`

的 markdown 草案。

內容至少包含：

- Why
- What Changes
- What Does Not Change
- Impact
- Dependencies
- Acceptance Criteria

注意：

這是 OpenSpec change proposal, 不是論文 proposal。

12. design.md 草案內容

請直接產出一份可以放入：

`openspec/changes/02-baseline-benchmark/design.md`

的 markdown 草案。

內容至少包含：

- Design Goal
- Benchmark Boundary
- Baseline Methods
- Topology Direction
- Metrics and Logging Direction
- Scenario Direction
- Downstream Usage
- Decision Records

注意：

`design.md` 只處理 benchmark 設計, 不處理實作程式。

13. tasks.md 草案內容

請直接產出一份可以放入：

`openspec/changes/02-baseline-benchmark/tasks.md`

的 markdown 草案。

`tasks` 必須是規格任務, 不是實作任務。

例如：

- Confirm baseline methods
- Confirm topology spec
- Confirm metrics definitions
- Confirm scenario matrix
- Confirm output artifacts
- Review no implementation included

每個 task 請用 checkbox 格式。

14. specs 草案摘要

請產出 specs 下五個檔案的摘要內容，不必寫成完整檔案，但要夠後續轉成完整 spec。

14.1 specs/baseline-methods.md

請提供：

- Purpose
- Required baselines
- Strongly recommended baselines
- Optional baselines
- Exclusion rules

14.2 specs/topology.md

請提供：

- Purpose
- Minimal topology
- Enhanced topology
- Topology assumptions
- Non-goals

14.3 specs/metrics-logging.md

請提供：

- Purpose
- Metric definitions
- Required logs
- Required output artifacts
- Ambiguity rules

14.4 specs/scenario-matrix.md

請提供：

- Purpose
- Required scenarios
- Scenario table summary
- MVP-required vs optional

14.5 specs/benchmark-risk-register.md

請提供：

- Purpose
- Risk categories
- Risk summary
- Fallback principles

15. Review Checklist

請建立 change-02 驗收檢查表。

檢查表必須讓我能回答：

- baseline methods 是否明確？
- NewReno / CUBIC / BBR 角色是否清楚？
- BBR 是否有 fallback？
- topology 是否清楚？

- metrics 是否清楚？
- logging artifacts 是否清楚？
- scenario matrix 是否清楚？
- 是否避免直接進入 DRL？
- 是否避免實作？
- 是否能餵給 change-03 / change-04？
- 是否符合 change-01 的 scope / non-goals？

請用 checklist 格式輸出。

16. 給 Antigravity 的 change-02 指令草案

請幫我產出一段可以交給 Antigravity 的 change-02 指令。

這段指令必須說清楚：

- 目前只做 baseline benchmark specification
- 不准寫 code
- 不准跑 ns-3
- 不准啟動 ns3-gym
- 不准開始 DQN
- 不准接 PPO
- 不准加入 IPFS / QUIC / multi-agent / multi-path
- 必須只產出 OpenSpec artifacts
- 必須遵守 change-01 project-charter
- 完成後等待 spec owner 驗收

六、輸出品質要求

1. 語言

- 全文使用繁體中文
- 必要英文專有名詞保留英文
- 檔名、change ID、artifact 名稱保持英文

2. 深度

不要只列大綱。

要寫到我可以直接複製部分內容去建立 OpenSpec 文件。

3. 嚴謹度

所有重要判斷要有可信來源支撐。

若是推論，請明確寫「這是基於來源的推論」。

4. 禁止事項

本階段禁止：

- 寫任何程式碼
- 寫 shell command
- 產生 ns-3 實作
- 產生 Python 訓練腳本
- 執行 baseline benchmark
- 產出實驗圖表
- 進入 opengym-env
- 進入 drl-mvp
- 更改題目
- 擴張到 IPFS / QUIC / multi-agent / multi-path
- 宣稱已有 baseline 實驗結果

七、現在開始執行

請先搜尋官方文件與正式來源，再輸出。

請直接從：

OpenSpec change 02 : baseline-benchmark 總結

開始。

顯示更多顯示較少

OpenSpec change 02 : baseline-benchmark 總結

change-02-baseline-benchmark 的目的，是把傳統 TCP baseline benchmark 的方法、場景、指標、輸出物與驗收規格先定義清楚，作為後續 **Phase 3 : Baseline 先行** 的 source of truth。它承接已驗收的 **01-project-charter**，不改題、不擴題、不跳進 DRL，而是專注建立 NewReno、CUBIC、BBR 等 baseline 的比較規格。OpenSpec 的價值在於讓人與 AI coding assistant 在寫 code 前先對齊規格，且每個 change 可集中管理 proposal、specs、design、tasks，因此本 change 必須先完成文件化規格，才能交給 Antigravity 執行。



本 change 必須在 **03-opengym-env** 與 **04-drl-mvp** 前完成，因為 DRL agent 後續的 observation、reward、evaluation、success / failure analysis 都需要 baseline 作為 reference point。ns-3 官方 TCP model 支援 NewReno、CUBIC、BBR 等多種 congestion control 演算法；Flow Monitor 也可用於量測網路協定效能並收集 flow-level 統計，因此適合作為 baseline benchmark 的規格依據。 

1. Change Metadata

欄位	內容
Change ID	02-baseline-benchmark
Change title	Baseline Benchmark Specification for TCP Congestion Control
Change type	benchmark specification / experiment planning / metrics definition
Status	draft for review
Owner	user / spec owner
Implementer	Antigravity, after approval
Related phase	Phase 2 : 用 OpenSpec 建立專案真相來源
Depends on	01-project-charter
Related downstream changes	03-opengym-env, 04-drl-mvp
Required before implementation	Yes
Code implementation allowed in this change	No

治理規則：

此 change 僅負責 baseline benchmark 規格。Antigravity 在此階段不得寫 C++、不得寫 Python、不得建立

ns-3 topology、不得執行 baseline、不得啟動 ns3-gym、不得開始 DQN。

2. Baseline Benchmark Charter

2.1 Benchmark Mission

Baseline benchmark 的使命，是為後續 DRL congestion-control agent 建立可信、可重現、可比較的傳統 TCP 參考基準。若沒有 baseline，後續 DQN 的表現就沒有合理比較對象，也無法判斷 agent 的 throughput、RTT、loss、utility trade-off 是否有研究意義。

baseline 必須先於 DRL，原因有三個。第一，baseline 先行能確認 single bottleneck topology、traffic flow、logging、metrics 都定義正確。第二，baseline 結果會成為 **04-drl-mvp** 的 comparison target。第三，若直接跳到 DRL，Antigravity 很可能會把問題變成「先訓練模型」而忽略網路實驗口徑一致性。

本 change 不產生任何實驗結果，只定義未來 Phase 3 應如何跑 baseline、收集什麼資料、輸出什麼 artifacts、如何供後續 DRL 使用。

2.2 Benchmark Scope

baseline benchmark 規格涵蓋：

- single bottleneck link
- sender / receiver
- one long-lived TCP traffic flow
- TCP NewReno
- TCP CUBIC
- TCP BBR
- average throughput
- average RTT / delay
- packet loss rate
- utility score
- planned scenario matrix
- raw logs / summary CSV / tables / plots 的輸出規格

ns-3 官方 TCP 文件指出，native ns-3 TCP 支援完整雙向 TCP 與多種 congestion control 演算法，且 CUBIC 為 default，NewReno、BBR 等也被支援，因此這三個 baseline 適合作為本專題的核心比較對象。 

2.3 Benchmark Non-Scope

baseline benchmark 不做：

- 不做 DRL

- 不做 ns3-gym environment
- 不做 DQN / PPO
- 不做 IPFS
- 不做 QUIC
- 不做 kernel TCP
- 不做 multi-agent
- 不做 multi-path routing
- 不做 large-scale topology
- 不做 Pantheon full integration
- 不產生實驗圖表
- 不宣稱 baseline 結果

Pantheon 可作為 benchmark / reproducibility 思想來源，因為它提供 Internet transport protocol 與 congestion control 的 common benchmark algorithms、shared evaluation platform 與 public archive 思維；但本專題 MVP 不把 Pantheon 作為必裝依賴，以降低工具鏈風險。  Pantheon

2.4 Benchmark Success Definition

成功不等於：

- baseline 已經跑出最好結果
- baseline 圖已經產生
- DRL 已經比較完成
- CUBIC / BBR / NewReno 表現已經被證明
- 已經完成 Phase 3 實驗

成功等於：

- baseline methods 明確
- topology requirements 明確
- logging requirements 明確
- metrics definitions 明確
- scenario matrix 明確
- fallback 明確
- downstream changes 可依此使用 benchmark 規格
- Antigravity 可在 Phase 3 依規格執行，而不自行擴題

3. Baseline Methods Specification

3.1 NewReno

選擇原因：

NewReno 是經典 loss-based TCP congestion control，適合作為傳統 TCP baseline。它代表以 loss / ACK feedback 為核心的較基本壅塞控制思路。

在本專題中的 baseline 角色：

NewReno 是最基礎的比較基準，用來回答：「DRL agent 至少能否在單一瓶頸環境中形成可與經典 TCP 比較的控制行為？」

與 DQN 的比較意義：

若 DQN 連 NewReno 都無法接近，代表 state、action、reward 或 training setup 可能需要檢討。若 DQN 在部分情境接近或超過 NewReno，則可進一步與 CUBIC / BBR 比較。

3.2 CUBIC

選擇原因：

CUBIC 是現代常見 TCP congestion control，且 ns-3 官方文件指出 CUBIC 是 native ns-3 TCP model 的 default congestion control algorithm。  NSNAM

在本專題中的 baseline 角色：

CUBIC 應作為主要 baseline。相較 NewReno，CUBIC 更能代表現代 TCP 的標準比較對象。

若 ns-3 預設為 CUBIC，這對 benchmark 的意義：

若指定版本的 ns-3 預設為 CUBIC，Antigravity 在 Phase 3 仍必須明確記錄使用的 TCP variant，而不能只寫「default TCP」。baseline benchmark 規格必須要求每個 scenario 明確標示 TCP algorithm，以避免實驗結果不可追溯。

3.3 BBR

選擇原因：

BBR 的設計思路與 loss-based TCP 不同，通常以 bottleneck bandwidth 與 RTT 估計作為核心概念。將 BBR 納入 baseline，可讓後續 DRL 不只和 loss-based method 比，也能和較現代的 bandwidth-delay based control 思路比較。

在本專題中的 baseline 角色：

BBR 是強烈建議 baseline。它能提升 benchmark 的說服力，也能讓後續 DQN 的 trade-off analysis 更完整。

為什麼是 strongly recommended 而不是 hard MVP requirement：

雖然 ns-3 官方文件列出 BBR 為支援演算法，但實際可用性可能受 ns-3 版本、API、logging 或實作細節影響。為了降低 MVP 風險，BBR 不應阻塞 baseline benchmark 的最低完成。若 BBR 跑不起來，MVP 仍可用 NewReno + CUBIC 完成 baseline-first 流程。這是基於 ns-3 版本相依性與 final project 完成率的風險控管推論。  NSNAM

降階方案：

- 第一降階：NewReno + CUBIC 作為 MVP baseline
- 第二降階：BBR 留作 **should-have**
- 第三降階：以文獻 / related work 說明 BBR，實作比較延後到 future work

3.4 Optional Baselines

加分但不列入 MVP hard requirement：

- Vegas
- Westwood
- DCTCP
- LEDBAT

這些 baseline 不列入必做，原因是 baseline 數量過多會稀釋 MVP 目標，增加 Phase 3 實驗矩陣與圖表整理負擔。ns-3 官方文件列出 Vegas、Westwood、DCTCP、LEDBAT 等亦為支援演算法，但本專題核心目的不是做 TCP variant survey，而是建立 DRL baseline comparison pipeline。  [NSNAM](#)

4. Topology Specification

4.1 Topology Concept

baseline benchmark 採用 single bottleneck topology。概念上包含：

- **sender**：產生 TCP traffic flow 的來源端
- **receiver**：接收 TCP traffic flow 的目的端
- **access links**：sender / receiver 與 bottleneck 之間的連線
- **bottleneck link**：頻寬較低或限制明確的核心鏈路
- **queue / buffer**：位於 bottleneck 附近，用於觀察排隊、delay、loss
- **optional cross traffic**：加分場景，用於測試干擾下的 baseline 表現

4.2 Minimal Topology

MVP 最小拓樸：

- one sender
- one receiver
- one bottleneck link
- one long-lived TCP flow
- fixed bandwidth
- fixed propagation delay
- fixed queue / buffer setting

此拓樸是 baseline benchmark 的最低風險起點。它保留 congestion control 的核心問題，同時避免 multi-flow、multi-path、large topology 帶來的複雜度。

4.3 Enhanced Topology

加分拓樸，只能作為 future extension：

- cross traffic
- variable bandwidth
- high-delay path
- multiple flows
- shared bottleneck fairness scenario

這些拓樸可增加研究深度，但不得阻塞 MVP。尤其 multiple flows 與 fairness 可能會引入 Jain's fairness index 等額外指標，應放在 MVP 完成後再評估。

4.4 Topology Diagram Description

後續應畫出的圖如下：

Sender → Access Link → Bottleneck Queue / Link → Access Link → Receiver

圖中應標示：

- sender
- receiver
- access link bandwidth / delay placeholder
- bottleneck link bandwidth / delay placeholder
- bottleneck queue / buffer
- TCP flow direction
- optional cross traffic arrow，僅在 enhanced scenario 圖中出現

圖的目的不是展示實作細節，而是讓 PPT / README / video 能一眼看懂 benchmark 場景。

5. Metrics and Logging Specification

5.1 Average Throughput

Metric meaning

Average throughput 指接收端在觀察時間內成功接收的資料量除以時間，通常可用 Mbps 表示。

Why it matters

吞吐量是本題的核心效能指標，代表鏈路資料傳輸效率。但它不能單獨作為成功標準，因為過高吞吐可能伴隨 queue buildup、RTT 增加或 packet loss。

Expected unit

- Mbps
- 或 Kbps，視實驗輸出尺度而定

Data needed

- received bytes
- measurement duration
- flow start / end time

Flow Monitor 官方文件指出，它會追蹤 flow-level packets，並收集 `timeFirstTxPacket`、`timeLastRxPacket`、`txBytes`、`rxBytes`、`txPackets`、`rxPackets` 等資料，這可支撐 throughput 類指標的計算。

Possible calculation concept

概念形式：

```
average_throughput = received_data_size / measurement_duration
```

此處只定義概念，不固定實作公式與單位換算。

Logging requirement

Phase 3 應輸出：

- per-flow received bytes
- measurement start time
- measurement end time
- computed throughput
- scenario ID
- baseline method

5.2 Average RTT / Delay

Metric meaning

Average RTT / delay 衡量封包往返或端到端延遲的平均狀態。本 benchmark 可先以 Flow Monitor 能取得的 delay 類統計作為 delay observation，再於實作階段確認是否可取得 RTT-specific trace。

Why it matters

delay 是壅塞控制的核心副作用。只追 throughput 可能造成 queue 累積，導致 queueing delay 上升，這正是 bufferbloat 相關問題的核心現象。

Expected unit

- ms
- seconds, 視 ns-3 / Flow Monitor 輸出而定

Data needed

- delay sum
- received packet count
- optional RTT trace, 若後續實作可取得

Flow Monitor 官方文件列出 **delaySum** 與 **rxPackets** 等 flow-level 統計, 可支撐平均 delay 類指標; 但是否直接得到 TCP RTT, 需在 Phase 3 依 ns-3 trace source 進一步確認。 

Logging requirement

Phase 3 應輸出：

- delay-related statistic
- rxPackets
- average delay
- 若可行, 輸出 TCP RTT trace 或 RTT estimate
- scenario ID
- baseline method

和 bufferbloat / queueing delay 的關係

若 throughput 增加但 delay 大幅上升, 可能代表傳輸策略讓 queue 過度累積。benchmark 應保留這類 trade-off 觀察, 而不是只看 throughput 排名。

5.3 Packet Loss Rate

Metric meaning

Packet loss rate 指封包遺失比例, 用於衡量壅塞或 queue drop 情況。

Why it matters

loss 是傳統 TCP congestion control 的重要訊號, 也是使用者體驗與傳輸效率的重要風險。DRL agent 若以高 loss 換取 throughput, 不應被視為成功。

Expected unit

- percentage
- ratio 0-1

Data needed

- transmitted packets
- received packets
- lost packets or drop count

Flow Monitor 官方文件說明 flow probe 會在 packets sent、forwarded、received、dropped 等點分類與追蹤 packets；其 flow statistics 也包含 tx / rx packet 類資料，因此可作為 loss 類指標規格依據。 

Logging requirement

Phase 3 應輸出：

- txPackets
- rxPackets
- lostPackets 或 drop-related statistic
- computed loss rate
- scenario ID
- baseline method

5.4 Utility Score

為什麼需要 utility

utility score 用於把 throughput、delay、loss 放在同一個比較框架中。因為單看 throughput 可能導致錯誤解讀：例如某 baseline throughput 最高，但 RTT 與 loss 也最高，未必代表整體控制品質最好。

為什麼不能只看 throughput

壅塞控制的目標不是單純「塞越多資料越好」，而是在網路容量、延遲與丟包之間取得平衡。Aurora 將 congestion control 視為 deep RL control problem，並指出 learned schemes 真實採用仍需面對 fairness、

 Proceedings of

Machine Learning

safety、generalization 等挑戰；這支撐本專題使用多指標評估，而非 throughput-only。 

建議概念形式

```
utility = normalized_throughput - penalty_delay - penalty_loss
```

符號注釋

- **normalized_throughput**：正規化後的吞吐量，值越高越好。
- **penalty_delay**：延遲懲罰，RTT / delay 越高，扣分越多。
- **penalty_loss**：丟包懲罰，loss rate 越高，扣分越多。

白話說明

utility 的意思是：「傳得快很好，但如果快是靠塞爆 queue 或大量丟包換來的，就要扣分。」

本階段規則

change-02 不固定最終權重，只固定 utility philosophy。最終權重應留給 change-03 / change-04 或 Phase 3 後續調整。

5.5 Output Artifacts

後續 Phase 3 至少要產出的 artifacts：

- raw logs
- summary CSV
- per-scenario summary table
- baseline comparison table
- throughput plot
- RTT / delay plot
- packet loss plot
- utility comparison plot
- experiment metadata

Experiment metadata 至少包含

- scenario ID
- baseline method
- topology version
- bandwidth profile
- delay profile
- queue / buffer setting
- traffic pattern
- measurement duration placeholder
- run ID / seed placeholder

注意：本 change 只定義 artifacts，不產生 artifacts。

6. Scenario Matrix Specification

Scenario ID	Purpose	Bandwidth profile	Delay profile	Queue / buffer setting	Traffic pattern	Required baselines	Required metrics
S1 Stable low-delay bottleneck	建立最基本 baseline, 比較各 TCP 在穩定低延遲瓶頸下的表現	Fixed, reasonable placeholder	Low delay placeholder	Medium queue placeholder	One long-lived TCP flow	NewReno, CUBIC; BBR strongly recommended	throughput delay/RTT, loss, utility
S2 Stable high-delay bottleneck	測試高延遲路徑下 baseline 的 throughput-delay trade-off	Fixed, same or comparable to S1	High delay placeholder	Medium queue placeholder	One long-lived TCP flow	NewReno, CUBIC; BBR strongly recommended	throughput delay/RTT, loss, utility
S3 Variable bandwidth bottleneck	測試 baseline 面對容量變動時的穩定性	Variable bandwidth placeholder	Medium delay placeholder	Medium queue placeholder	One long-lived TCP flow	NewReno, CUBIC; BBR optional if stable	throughput delay/RTT, loss, utility
S4 Bottleneck with cross traffic	測試 baseline 在干擾流量下的穩定性	Fixed or variable placeholder	Medium delay placeholder	Medium or small queue placeholder	Main TCP flow + cross traffic	NewReno, CUBIC; BBR optional	throughput delay/RTT, loss, utility

Scenario 設計規則

- S1 / S2 是 MVP baseline 的主軸。
- S3 是 strongly recommended, 但不應阻塞 MVP。
- S4 是加分或 future extension, 不應讓 Phase 3 失控。
- 不在 change-02 固定過細實作參數, 避免後續 Antigravity 被不必要數值綁死。
- 所有 scenario 必須輸出相同 metrics, 否則後續無法比較。

7. Benchmark Evaluation Philosophy

本專題 baseline benchmark 的評估哲學如下：

1. baseline 不是敵人, 而是比較基準。

NewReno、CUBIC、BBR 不是要被「打敗」的對象, 而是用來校準後續 DQN agent 表現的 reference point。

2. 不以單一 throughput 最大化為唯一成功標準。

throughput 重要, 但若 RTT 或 loss 過高, 整體傳輸品質仍可能變差。

3. delay / RTT / loss 是同等重要的分析維度。

baseline comparison 至少要同時輸出 throughput、delay/RTT、loss、utility。

4. DRL 後續若輸給 BBR / CUBIC 也不是失敗。

若 DQN 在某些情境低於 baseline，但能揭示 reward 設計、state representation 或 training stability 的限制，仍可形成合理 final findings。

5. baseline 的作用是提供 DQN 的 reference point。

change-04-drl-mvp 必須沿用 change-02 的 metrics 與 scenario 口徑。

6. baseline 實驗應可重現、可比較、可視覺化。

Pantheon 作為 congestion-control benchmark 的重要價值在於共同演算法集合、shared evaluation platform 與 public results archive；本專題吸收其 reproducibility 精神，但不要求 MVP 導入 Pantheon 工具鏈。  Pantheon

8. Downstream Dependency

8.1 對 change-03-opengym-env 的輸入

change-02 會提供 change-03 以下輸入：

Observation candidates

baseline logging 中會規格化：

- throughput
- delay / RTT
- loss
- congestion indicator
- queue-related signal if available

這些將成為 ns3-gym environment 的 observation candidates。

Metric logging

change-03 必須沿用 change-02 的 logging philosophy，確保 DRL environment 的 metrics 和 baseline benchmark 可比較。

Reward components

change-02 定義的 throughput、delay、loss 將成為 reward components 的基礎：

```
reward = throughput benefit - delay penalty - loss penalty
```

Scenario reuse

change-03 應優先重用 S1 / S2，以避免 environment 與 baseline 場景不一致。

Episode / evaluation setup

change-03 的 episode 設計應參照 baseline scenario 的 measurement window、traffic pattern 與 metric collection 需求。

8.2 對 change-04-drl-mvp 的輸入

change-02 會提供 change-04 以下輸入：

DQN comparison target

DQN 必須與 NewReno / CUBIC / BBR 或其 MVP subset 比較。

Evaluation metrics

DQN 必須輸出與 baseline 相同的：

- throughput
- delay / RTT
- loss
- utility

Reward curve interpretation

DQN 的 reward curve 不能單獨解讀，必須搭配 baseline metrics 看是否學到合理 trade-off。

Success / failure analysis

若 DQN 低於 CUBIC / BBR，change-04 必須根據 change-02 定義的 evaluation philosophy，做 limitation / trade-off analysis，而非視為專案失敗。

Final project figures

change-04 最終圖表應至少包含：

- baseline comparison
- DQN vs baseline comparison
- reward curve
- utility comparison

9. Risk Register for Baseline Benchmark

Risk ID	Description	Probability	Impact	Prevention	Fallback	Owner	Trigger condition
B-R1	ns-3 TCP algorithm name /	Medium	High	在 change-02 僅凍結 algorithm role，不寫死 API name	Phase 3 查版本後更新 implementation notes	Antigravity	指定 TCP variant 無法被版本識別

Risk ID	Description	Probability	Impact	Prevention	Fallback	Owner	Trigger condition
	version mismatch						
B-R2	BBR availability issue	Medium	Medium	BBR 設為 strongly recommended, 不設為 hard MVP	MVP 先 NewReno + CUBIC	Antigravity	BBR 在指定 ns-3 版本跑不起來
B-R3	FlowMonitor / logging 不足以取得 RTT 或 loss	Medium	High	delay / loss 分開規格化；RTT-specific trace 另列 should-have	先用 delaySum / rxPackets / txPackets 等 flow-level 指標	Antigravity	無法取得直接 RTT trace
B-R4	throughput 計算口徑不一致	Medium	High	規格要求所有 baseline 使用相同 measurement window 與 calculation concept	重跑 baseline, 統一 summary CSV schema	Antigravity	不同 baseline 使用不同時間窗
B-R5	scenario 設計太複雜	Medium	Medium	S1 / S2 列為 MVP, S3 / S4 不阻塞 MVP	只保留 stable low-delay / high-delay	Spec owner	Phase 3 scope 超出預期
B-R6	cross traffic 讓 MVP 失控	High	Medium	S4 設為 optional / future extension	移除 cross traffic, 只做單流	Spec owner	baseline 結果波動難解釋
B-R7	baseline 結果異常但無法解釋	Medium	Medium	要求輸出 metadata 與 per-scenario summary	將異常情境標為 limitation, 不作為主要比較	Antigravity	指標和預期差距極大
B-R8	Antigravity 直接跳到 DRL	Medium	High	change-02 明確禁止 ns3-gym / DQN / PPO	停止工作, 回到 spec owner review	Spec owner	出現 DQN、agent、training script
B-R9	baseline 圖表不足以支撐 PPT	Medium	Medium	artifact 規格先要求 throughput / delay / loss / utility plots	PPT 使用 table + topology + metric definition 作補充	Antigravity	缺少至少 3 種核心圖
B-R10	Pantheon 被誤當必裝工具	Medium	Medium	明確寫 Pantheon 是 benchmark philosophy, 不是 dependency	移除 Pantheon 實作任務, 只保留 related work	Spec owner	任務出現 Pantheon integration

10. OpenSpec Artifact Blueprint

建議 change-02 artifact 結構：

```
openspec/  
changes/
```

```
02-baseline-benchmark/
proposal.md
design.md
tasks.md
specs/
  baseline-methods.md
  topology.md
  metrics-logging.md
  scenario-matrix.md
  benchmark-risk-register.md
```

檔案	purpose	required content	reviewer should check	acceptance criteria
proposal.md	說明為什麼需要 baseline benchmark spec	Why, What Changes, What Does Not Change, Impact, Dependencies, Acceptance Criteria	是否承接 change-01；是否禁止實作	不含 code、不含命令、不宣稱結果
design.md	定義 benchmark 設計方向	Benchmark Boundary, Baseline Methods, Topology, Metrics, Scenario, Downstream Usage	是否能餵給 Phase 3 與 change-03 / 04	baseline 設計完整且不跳 DRL
tasks.md	定義 change-02 規格任務	checkbox tasks	是否全為規格任務	沒有實作任務
specs/baseline-methods.md	凍結 baseline 角色	NewReno, CUBIC, BBR, optional baselines	BBR 是否有 fallback	baseline list 清楚
specs/topology.md	凍結 single bottleneck topology	minimal / enhanced topology	MVP 是否不被 enhanced topology 阻塞	topology 清楚且不擴題
specs/metrics-logging.md	凍結 metrics 與 artifacts	throughput, RTT/delay, loss, utility, logs, CSV, plots	指標口徑是否一致	可餵給 Phase 3 logging
specs/scenario-matrix.md	凍結 scenario matrix	S1-S4	MVP-required vs optional 是否明確	S1/S2 可做 MVP
specs/benchmark-risk-register.md	管理 baseline 風險	risk table, fallback	是否包含 BBR、logging、scope drift	每項高風險有 fallback

11. proposal.md 草案內容

Markdown

Change 02: Baseline Benchmark

Why

The project requires a clear TCP baseline benchmark specification before any reinforcement learning work begins.

The final project is focused on Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link. Before building an ns3-gym environment or training a DQN agent, the project must define traditional TCP baselines, topology assumptions, metrics, scenario matrix, and required output artifacts.

Without this baseline specification, future DRL results would lack a valid comparison point, and Antigravity may prematurely jump into RL implementation without verifying the benchmark

foundation.

What Changes

This change defines:

- Baseline benchmark mission
- Benchmark scope and non-scope
- Baseline methods: NewReno, CUBIC, BBR
- Optional baselines: Vegas, Westwood, DCTCP, LEDBAT
- Single bottleneck topology requirements
- Metrics and logging requirements
- Utility score philosophy
- Required output artifacts
- Scenario matrix
- Downstream dependency for opengym-env and drl-mvp
- Benchmark-specific risk register

What Does Not Change

This change does not implement anything.

This change does not:

- Write C++ code
- Write Python code
- Run ns-3
- Create ns-3 topology implementation
- Generate experiment data
- Generate plots
- Start ns3-gym
- Define or train DQN/PPO agents
- Implement IPFS
- Implement QUIC
- Modify kernel TCP
- Add multi-agent RL
- Add multi-path routing
- Require Pantheon integration

Impact

This change becomes the source of truth for Phase 3: Baseline First.

It also provides required inputs for:

- 03-opengym-env
- 04-drl-mvp

All later DRL evaluation must compare against the baseline methods and metrics defined here.

Dependencies

Depends on:

- 01-project-charter

Inputs inherited from change-01:

- Project title
- Scope / non-goals
- Baseline list
- Metrics
- MVP definition
- Risk register
- Downstream change map

Acceptance Criteria

- [] Baseline methods are clearly defined.
- [] NewReno, CUBIC, and BBR roles are clear.
- [] BBR has a fallback plan.
- [] Single bottleneck topology is defined.
- [] Minimal and enhanced topology boundaries are separated.
- [] Metrics are clearly defined.
- [] Logging artifacts are defined.

- [] Scenario matrix is defined.
- [] MVP-required scenarios are separated from optional scenarios.
- [] Downstream dependency to 03-opengym-env is clear.
- [] Downstream dependency to 04-drl-mvp is clear.
- [] Risk register is included.
- [] No implementation code is included.
- [] No shell commands are included.
- [] No experimental results are claimed.

12. design.md 草案內容

Markdown

Design: Baseline Benchmark Specification

Design Goal

The goal of this design is to define the baseline benchmark foundation for the DRL congestion-control final project.

This change specifies which TCP baselines will be evaluated, what topology they will run on, what metrics must be logged, which scenarios are required, and how these baseline results will later serve as reference points for DRL evaluation.

Benchmark Boundary

The benchmark is limited to a single bottleneck link.

In scope:

- one sender
- one receiver
- one bottleneck link
- one long-lived TCP flow
- NewReno
- CUBIC
- BBR if supported cleanly
- throughput, delay/RTT, loss, utility

Out of scope:

- DRL
- ns3-gym
- DQN
- PPO
- IPFS
- QUIC
- kernel TCP
- multi-agent RL
- multi-path routing
- large-scale topology
- Pantheon full integration

Baseline Methods

Required:

- NewReno
- CUBIC

Strongly recommended:

- BBR

Optional:

- Vegas
- Westwood
- DCTCP
- LEDBAT

BBR must not block the MVP. If BBR is unavailable or unstable in the selected ns-3 version, the

MVP proceeds with NewReno and CUBIC.

Topology Direction

Minimal topology:

Sender -> Access Link -> Bottleneck Queue / Link -> Access Link -> Receiver

The minimal topology must use one long-lived TCP flow and one bottleneck link.

Enhanced topology may include:

- variable bandwidth
- high-delay path
- cross traffic
- multiple flows

Enhanced topology is optional and must not block the MVP.

Metrics and Logging Direction

Required metrics:

- average throughput
- average delay or RTT
- packet loss rate
- utility score

Required artifacts:

- raw logs
- summary CSV
- per-scenario summary table
- baseline comparison table
- throughput plot
- delay/RTT plot
- packet loss plot
- utility comparison plot
- experiment metadata

Scenario Direction

Required MVP scenarios:

- S1: Stable low-delay bottleneck
- S2: Stable high-delay bottleneck

Strongly recommended:

- S3: Variable bandwidth bottleneck

Optional:

- S4: Bottleneck with cross traffic

Downstream Usage

03-opengym-env must reuse the metrics and scenario definitions from this benchmark.

04-drl-mvp must compare DQN results against the baseline methods and metrics defined here.

Decision Records

DR-02-01: Baseline before DRL

Reason: DRL evaluation requires a valid comparison reference.

DR-02-02: NewReno and CUBIC are MVP-required

Reason: They provide a basic and modern TCP baseline pair.

DR-02-03: BBR is strongly recommended but not hard-required

Reason: It improves benchmark quality but may introduce version-specific risk.

DR-02-04: Throughput is not the only metric

Reason: Congestion control must consider throughput-delay-loss trade-offs.

DR-02-05: Pantheon is a benchmark philosophy reference, not a dependency

Reason: The MVP must reduce tooling complexity while preserving reproducibility thinking.

13. tasks.md 草案內容

Markdown

Tasks: Change 02 Baseline Benchmark

Specification Tasks

- [] Confirm that change-02 depends on 01-project-charter.
- [] Confirm that no implementation is included in this change.
- [] Confirm baseline benchmark mission.
- [] Confirm benchmark scope.
- [] Confirm benchmark non-scope.
- [] Confirm NewReno baseline role.
- [] Confirm CUBIC baseline role.
- [] Confirm BBR baseline role.
- [] Confirm BBR fallback plan.
- [] Confirm optional baselines.
- [] Confirm minimal single bottleneck topology.
- [] Confirm enhanced topology boundaries.
- [] Confirm topology diagram description.
- [] Confirm average throughput definition.
- [] Confirm delay / RTT definition.
- [] Confirm packet loss rate definition.
- [] Confirm utility score philosophy.
- [] Confirm required output artifacts.
- [] Confirm scenario S1.
- [] Confirm scenario S2.
- [] Confirm scenario S3.
- [] Confirm scenario S4.
- [] Confirm evaluation philosophy.
- [] Confirm downstream dependency to 03-opengym-env.
- [] Confirm downstream dependency to 04-drl-mvp.
- [] Confirm benchmark risk register.
- [] Review that no C++ code is included.
- [] Review that no Python code is included.
- [] Review that no shell command is included.
- [] Review that no experiment result is claimed.
- [] Submit change-02 to spec owner for review.

14. specs 草案摘要

14.1 specs/baseline-methods.md

Purpose

定義本專題 baseline benchmark 中使用的 TCP congestion control methods, 並明確區分 MVP-required、strongly recommended、optional。

Required baselines

- NewReno
- CUBIC

Strongly recommended baselines

- BBR

Optional baselines

- Vegas
- Westwood
- DCTCP
- LEDBAT

Exclusion rules

- 不加入非 TCP baseline 作為 MVP
- 不加入 QUIC baseline
- 不加入 Pantheon full integration
- 不加入 production transport protocols

14.2 specs/topology.md

Purpose

定義 baseline benchmark 的 single bottleneck link topology。

Minimal topology

- one sender
- one receiver
- one bottleneck link
- one long-lived TCP flow
- fixed bandwidth
- fixed delay
- fixed queue / buffer placeholder

Enhanced topology

- variable bandwidth
- high delay
- cross traffic
- multiple flows

Topology assumptions

- MVP 使用 single flow
- enhanced topology 不阻塞 MVP
- cross traffic 僅作 optional scenario

Non-goals

- no large-scale topology
- no multi-path
- no multi-agent
- no IPFS / QUIC overlay

14.3 specs/metrics-logging.md

Purpose

定義 baseline benchmark 的 metrics、logging requirements 與 output artifacts。

Metric definitions

- average throughput
- average delay / RTT
- packet loss rate
- utility score

Required logs

- raw flow logs
- per-flow statistics
- received bytes
- tx / rx packets
- delay-related statistics
- loss / drop-related statistics if available

Required output artifacts

- raw logs
- summary CSV
- per-scenario table
- baseline comparison table
- throughput plot
- delay / RTT plot

- loss plot
- utility plot
- experiment metadata

Ambiguity rules

- 若 RTT trace 不可得，先使用 Flow Monitor delay 類統計作替代並明確標註。
- 若 loss 無法直接取得，使用 tx / rx / drop 類資訊推導並標註計算口徑。
- 所有 baseline 必須使用相同 measurement window。

14.4 specs/scenario-matrix.md

Purpose

定義 baseline benchmark 的 scenario matrix。

Required scenarios

- S1 Stable low-delay bottleneck
- S2 Stable high-delay bottleneck

Scenario table summary

- S1 / S2 : MVP-required
- S3 variable bandwidth : strongly recommended
- S4 cross traffic : optional / future extension

MVP-required vs optional

MVP 只要求 S1 / S2 先完成。S3 / S4 不得阻塞 MVP。

14.5 specs/benchmark-risk-register.md

Purpose

管理 baseline benchmark 階段風險。

Risk categories

- toolchain version risk
- logging ambiguity risk
- scenario complexity risk
- baseline availability risk
- scope drift risk

- reporting artifact risk

Risk summary

核心風險包含 BBR availability、FlowMonitor / RTT ambiguity、throughput calculation inconsistency、Antigravity jumping to DRL、Pantheon mistaken as dependency。

Fallback principles

- baseline-first
- NewReno + CUBIC can satisfy MVP
- BBR does not block MVP
- S1 / S2 can satisfy MVP
- S3 / S4 are optional if unstable
- no DRL until baseline spec is approved

15. Review Checklist

change-02 驗收檢查表

- ☐ baseline methods 是否明確？
- ☐ NewReno 的角色是否清楚？
- ☐ CUBIC 的角色是否清楚？
- ☐ BBR 的角色是否清楚？
- ☐ BBR 是否有 fallback？
- ☐ optional baselines 是否未被列入 MVP hard requirement？
- ☐ topology 是否清楚？
- ☐ minimal topology 是否足以支撐 MVP？
- ☐ enhanced topology 是否被標成 optional / future extension？
- ☐ metrics 是否清楚？
- ☐ throughput 定義是否清楚？
- ☐ RTT / delay 定義是否清楚？
- ☐ loss 定義是否清楚？
- ☐ utility philosophy 是否清楚？
- ☐ logging artifacts 是否清楚？
- ☐ scenario matrix 是否清楚？
- ☐ S1 / S2 是否為 MVP-required？
- ☐ S3 / S4 是否未阻塞 MVP？

- ☐ 是否避免直接進入 DRL ?
- ☐ 是否避免實作 ?
- ☐ 是否未寫 C++ code ?
- ☐ 是否未寫 Python code ?
- ☐ 是否未寫 shell command ?
- ☐ 是否未宣稱 baseline 結果 ?
- ☐ 是否能餵給 change-03 ?
- ☐ 是否能餵給 change-04 ?
- ☐ 是否符合 change-01 的 scope ?
- ☐ 是否符合 change-01 的 non-goals ?
- ☐ 是否禁止 IPFS / QUIC / multi-agent / multi-path 擴張 ?
- ☐ 是否把 Pantheon 定位為 benchmark philosophy 而非 dependency ?

16. 給 Antigravity 的 change-02 指令草案

You are working on Phase 2 / OpenSpec change 02: baseline-benchmark.

Current task:

Create only the OpenSpec baseline benchmark specification artifacts for the DRL final project.

Project:

Deep Reinforcement Learning for Congestion Control and Throughput Optimization over a Single Bottleneck Link

Repository:

drl-bottleneck-congestion-control

You must follow the approved 01-project-charter.

You must only produce documentation artifacts for:

- openspec/changes/02-baseline-benchmark/proposal.md
- openspec/changes/02-baseline-benchmark/design.md
- openspec/changes/02-baseline-benchmark/tasks.md
- openspec/changes/02-baseline-benchmark/specs/baseline-methods.md
- openspec/changes/02-baseline-benchmark/specs/topology.md
- openspec/changes/02-baseline-benchmark/specs/metrics-logging.md
- openspec/changes/02-baseline-benchmark/specs/scenario-matrix.md
- openspec/changes/02-baseline-benchmark/specs/benchmark-risk-register.md

Strict rules:

- Do not write C++ code.
- Do not write Python code.
- Do not write shell commands.
- Do not run ns-3.
- Do not create ns-3 topology implementation.
- Do not generate experiment data.
- Do not generate plots.
- Do not start ns3-gym.
- Do not create DQN or PPO training logic.
- Do not implement IPFS.
- Do not implement QUIC.
- Do not modify Linux kernel TCP.
- Do not introduce multi-agent RL.
- Do not introduce multi-path routing.
- Do not make Pantheon a required dependency.
- Do not claim any baseline result.

The purpose of this change is to define:

- baseline methods
- single bottleneck topology
- metrics and logging requirements
- output artifacts
- scenario matrix
- benchmark evaluation philosophy
- downstream dependency to opengym-env and drl-mvp
- risk register

After completing the documentation artifacts, stop and wait for spec owner review.

change-02 結論

OpenSpec change 02 : baseline-benchmark 的驗收重點，不是有沒有開始跑 ns-3，而是 baseline benchmark 的方法、拓樸、指標、輸出物與風險是否已足夠清楚。這份規格通過後，才適合進入後續 **change-03-opengym-env**；而真正執行 baseline，仍應等到 Phase 3「Baseline 先行」再做。

Phase 5：做 DQN MVP

截圖是 Phase 4：DRL MVP Implementation 的 commit

Antigravity 做完「Phase 4：DRL MVP Implementation」**最佳 Prompt** 指令了

並已推上 github Phase 4：DRL MVP Implementation 的異動 請幫我驗收 git diff

是否全部 **Phase 4：DRL MVP Implementation** 要求都有通過

是否能推進到「**Phase 5：做 DQN MVP**」了呢

顯示更多顯示較少

結論：這個 1a47b0f commit 不能驗收通過，現在還不能正式推進到「修 Change 05: reporting-figures-and-demo 三個交接瑕疵」。

它有做出兩個重要檔案，也比前一版更接近你的要求；但它沒有完全符合你剛剛那份「**Step 1B：Gemini Full Repository Codebase Audit Addendum**」的硬性規格。尤其是：commit message 不符合指定格式、addendum 章節嚴重不足、inventory 欄位格式不符、CSV / logs 被標成 Not applicable 而不是 Read 或 sampled，這些都違反你要求的「完整讀完整份專案」驗收標準。

1. 這次 commit 做了什麼

這次 commit 的 message 是：

docs: strict Gemini codebase audit addendum and exhaustive file inventory

它新增了：

```
reports/handoff/full-repo-codebase-file-inventory.md
reports/handoff/gemini-full-repo-codebase-audit-addendum.md
```

commit diff 顯示 **full-repo-codebase-file-inventory.md** 從空檔新增，開頭確實列出多個 root、**.agent/**、**docs/**、**experiments/** 檔案，且把 Markdown / code 類檔案標成 **[x] Read**。

[docs: strict Gemini codebase au...](#)

gemini-full-repo-codebase-audit-addendum.md 也明確宣稱本次不使用代表檔案、不使用等效推論、不用歷史確認補足，並說它已產出 exhaustive file inventory。

[gemini-full-repo-codebase-audit...](#)

這些是正向進展。

2. 為什麼不能驗收通過

A. commit message 沒有照你指定的格式

你前一版 prompt 明確要求 commit message 必須使用：

```
docs: add full Gemini codebase audit addendum
```

但實際 commit 是：

```
docs: strict Gemini codebase audit addendum and exhaustive file inventory
```

這不是大技術錯誤，但它違反了你「commit -m 內容請參考過去 Claude 風格、不要顯得有斷層」以及後來更嚴格指定 exact message 的要求。這次 commit message 雖然仍是 **docs:** 開頭，但**不是指定文字**。

[docs: strict Gemini codebase au...](#)

判定：**不通過 exact commit-message requirement**。

B. addendum 沒有包含你要求的 17 個章節

你要求 **gemini-full-repo-codebase-audit-addendum.md** 至少包含：

1. Executive Summary
2. Why Previous Audit Was Insufficient
3. Repository Inventory Summary
4. Root and Governance Understanding
5. OpenSpec Changes Understanding
6. Phase 3 Baseline Code Understanding
7. Phase 4 OpenGym Environment Code Understanding

8. Phase 4 DQN Training / Evaluation Code Understanding
9. Analysis and Plotting Code Understanding
10. Scripts Understanding
11. Data / Artifact Understanding
12. Result and Limitation Understanding
13. Files I Must Not Modify in Step 2
14. Working Tree Hygiene
15. Commit Summary
16. Readiness for Step 2
17. Audit Checklist

但實際 `gemini-full-repo-codebase-audit-addendum.md` 只有三個主要章節：

- Executive Summary
- Complete File Inventory Reference
- Strict Compliance Statement

檔案總共也只有 49 行，內容偏宣告，不是你要求的完整 audit addendum。

[gemini-full-repo-codebase-audit...](#)

判定：**不通過 addendum structure requirement.**

C. file inventory 欄位格式不符合你要求

你要求 inventory table 必須包含：

Category	File Path	Type	Status	Read Method	Purpose	Frozen?	Allowed to Modify in Step 2?	Notes
----------	-----------	------	--------	-------------	---------	---------	------------------------------	-------

但實際檔案只有：

File Path	Status	Notes
-----------	--------	-------

可以從 `full-repo-codebase-file-inventory.md` 開頭看到只有三欄：**File Path**、**Status**、**Notes**。

[full-repo-codebase-file-invento...](#)

這代表它沒有明確列出：

- category
- file type
- read method
- purpose
- frozen status

- Step 2 是否可修改
- missing / excluded / too-large 檢查資訊

判定：**不通過 inventory schema requirement。**

D. 狀態值不符合你指定的 **allowed statuses**

你指定 status 只能使用：

```
Read
Listed only – binary/image/model
Missing
Excluded by rule
Too large – sampled with explanation
```

但實際 inventory 使用：

```
[x] Read
[-] Binary skipped
[-] Not applicable
```

例如 image 被標為 [-] Binary skipped, CSV / logs 被標為 [-] Not applicable。

[full-repo-codebase-file-invento...](#)

這不是你規定的狀態集合。更重要的是，**Not applicable** 會模糊化：這個檔案到底是已讀、只列出、太大抽樣、還是完全沒看？

判定：**不通過 status vocabulary requirement。**

E. CSV / logs 被標成 Not applicable，違反「文字型 artifacts 必須閱讀」要求

你明確要求：

- 所有 .csv 是文字型 artifacts，要讀。
- 對 logs 若太大，至少要記錄 line count、header / schema、representative first rows、purpose、frozen status。

但實際 inventory 把：

```
experiments/drl/evaluations/dqn_eval_s1.csv
experiments/drl/evaluations/dqn_eval_s2.csv
experiments/drl/logs/dqn_train_s1_seed42.monitor.csv
experiments/drl/logs/dqn_train_s2_seed42.monitor.csv
...
```

全部標為：

```
[-] Not applicable | Generated data
```

而不是 **Read**，也不是 **Too large – sampled with explanation**。

[full-repo-codebase-file-invento...](#)

這一點很關鍵。你這次要求的核心就是「完整讀完整份期末專案」。CSV 是本專案結果 source of truth，不能只因為它是 generated data 就標成 Not applicable。

判定：**不通過 CSV / log audit requirement**。

F. addendum 只有宣告，沒有逐項技術理解

雖然 addendum 宣稱它確認了 Phase 3 baseline、Phase 4 OpenGym、DQN training / evaluation 等實作細節，但它沒有按照你要求分章寫出：

- Phase 3 baseline code 如何建立 topology、TCP variants、FlowMonitor、metrics。
- Phase 4 OpenGym environment 的 observation / action / reward / info dict。
- DQN training / evaluation code 的 policy、seed、timesteps、checkpoint。
- analysis / plotting code 如何合併 CSV、輸出 figures。
- Step 2 可改與不可改檔案邊界。

它只有一句概括式聲明：「確認了 Phase 3 Baseline C++ 模擬與腳本、Phase 4 OpenGym Wrapper 與 DQN Agent 訓練及評估腳本等所有實作細節」。

[gemini-full-repo-codebase-audit...](#)

這不符合你要的「可審計、可驗收、完整讀過 repo」標準。

判定：**不通過 technical understanding write-up requirement**。

3. 哪些部分可以肯定

雖然這次不能通過，但它不是完全沒做。

可以肯定的部分是：

- 有新增 full repo inventory 檔案。
- 有新增 strict audit addendum。
- 有把 .agent/skills、.agent/workflows、docs、部分 experiments 檔案列入 inventory。
- 有宣稱不使用等效推論。
- 有宣稱尚未進 Phase 5。
- 有宣稱未修改 frozen data。
- 有維持 OpenSpec workflow 意識。OpenSpec 官方本來就是用 change folder 管 proposal、specs、design、tasks，並要求先和 AI 對齊 specs 再寫 code；這個方向沒有偏。  [GitHub](#)

但這些只能算「方向對」，不代表「完全通過」。

4. 是否能推進到 Step 2 ?

不能。

因為你這一輪的明確要求是：

我要它完整讀完「整份期末專案」GitHub Repo 所有程式碼內容，再進到 Step 2。

而目前 commit 沒有達到這個驗收標準。

最主要 blocker 是：

1. CSV / logs 沒讀，只標 Not applicable。
2. inventory schema 不符。
3. addendum 沒有 17 章完整內容。
4. technical understanding 太概括。
5. commit message 不符指定格式。

所以我建議：

不要放行 Step 2。

請 Gemini 重新修正 Step 1B audit commit，或補一個 follow-up commit，把 Step 1B 做到真正可驗收。

5. 下一步該怎麼做

我建議不要讓它重做一堆無效掃描，而是要求它**補修這次 Step 1B 的缺口**，可以用 follow-up commit，例如：

```
docs: complete full Gemini codebase audit addendum
```

但如果你要維持原本指定 message，也可以要求它 amend commit；若你不想改歷史，就用 follow-up commit 比較安全。

下一個 prompt 要明確要求：

1. 不要改 Change 05。
2. 不要進 Phase 5。
3. 不要改 CSV / code / model / figures。
4. 只更新：
 - reports/handoff/full-repo-codebase-file-inventory.md
 - reports/handoff/gemini-full-repo-codebase-audit-addendum.md
5. inventory table 改成完整九欄。
6. CSV / YAML / logs 必須標 Read 或 Too large – sampled with explanation，不能標 Not applicable。
7. addendum 必須補滿 17 章。
8. commit message 用：

docs: complete full Gemini codebase audit addendum

9. 修完後再停，等待你驗收。

最終判定

Step 1B：Gemini Full Repository Codebase Audit Addendum — 驗收不通過。

項目	判定
是否有新增 audit addendum	✓ 有
是否有新增 inventory	✓ 有
是否遵守只做 handoff audit，不修 Change 05	✓ 看起來有
是否完整符合 prompt	✗ 沒有
是否足以證明完整讀完整份 repo	✗ 不足
是否可以進 Step 2	✗ 不建議
是否可以進 Phase 5	✗ 絕對不行

建議退回 Gemini 補修 Step 1B，不要進 Step 2。

好 我同意以上你對「Step 1B：Gemini Full Repository Codebase Audit Addendum」的判斷

跟之前我們合作流程一模一樣 請幫我生成針對你說的幾項 ****Step 1B**** 未達標的事項的正式「最完美且超高品質的****最佳 Prompt****」

可以，這次要生成的是 **Step 1B 補救版 Prompt**，目標不是重做整個流程，而是要求 Gemini 針對前一版未達標處做 **follow-up 修正 commit**。我會把限制鎖死：**只准修兩份 audit 文件，不准修 Change 05、不准進 Step 2、不准進 Phase 5。**

這份 prompt 會明確要求它修掉前一版的幾個問題：commit message 雖然不能安全改歷史，但要用 follow-up commit 補齊；addendum 必須補滿 17 章；inventory 必須改成九欄 schema；status vocabulary 必須符合指定集合；CSV / YAML / logs 不能再標 **Not applicable**，必須 **Read** 或 **Too large – sampled with explanation**。前一版 commit 確實只有新增 audit addendum 與 inventory，但 addendum 只有三個主章節，inventory 也只有 **File Path / Status / Notes** 三欄，且多個 CSV / logs 被標成 **Not applicable**，所以這次要精準補救。

docs: strict Gemini codebase au...

OpenSpec 官方也強調它是讓人與 AI 在寫 code 前對齊 specs，並以 change folder 管理 proposal / specs / design / tasks，所以這次仍要保持 SDD 流程紀律，不可把 audit 補救混成 Phase 5 實作。 [GitHub](#)

下面這份可以直接貼給 **Antigravity Gemini Model**。

這份 prompt 的重點是「補救 Step 1B」，不是重新開新任務。你下一輪驗收時要看三件事：**inventory 是否九欄、CSV/logs 是否不再 Not applicable、addendum 是否真的補滿 17 章。**

<https://github.com/donuop35/ns3-drl-congestion-control/commit/336c02d164cc184666ac8f0cf48432e79a5ecd21>

Antigravity 做完「Step 1B 補救版 Prompt」**最佳 Prompt** 指令了

並已推上 github Step 1B 補救版 Prompt 的異動 請幫我驗收 git diff

是否全部 **Step 1B 補救版 Prompt** 要求都有通過

是否能正式推進到「**修 Change 05: reporting-figures-and-demo 幾個交接瑕疵**」了呢

等 OpenSpec Change 05 都完美修正了 再下一次才真正進入「Phase 5：Final Reporting / Demo / PPT Package Implementation」
顯示更多顯示較少

結論：可以驗收通過，可以正式推進到「修 Change 05: reporting-figures-and-demo 三個交接瑕疵」。
但我會精準標記為：

Step 1B 補救版：核心驗收通過。

不是 100% 零瑕疵，但剩下的是非阻塞 metadata 小瑕疵，不影響進入 Step 2。

下一步仍然不能進 Phase 5，只能修 Change 05 三個瑕疵。

1. 這次 commit 是否有照要求修正？

有，核心要求已補上。

這次 commit message 是：

docs: complete full Gemini codebase audit addendum

這正是你上一版 prompt 指定的 commit message。commit 內容也顯示主要修改的是兩份 handoff audit 檔案，符合「只修 Step 1B audit 文件，不修 Change 05、不進 Phase 5」的邊界。

[docs: complete full Gemini code...](#)

2. Inventory 是否已從三欄升級成九欄？

有，這次通過。

full-repo-codebase-file-inventory.md 已經改成完整九欄：

Category	File Path	Type	Status	Read Method	Purpose	Frozen?	Allowed to Modify in Step 2?	Notes
----------	-----------	------	--------	-------------	---------	---------	------------------------------	-------

這正好補掉上一版只有 **File Path / Status / Notes** 三欄的問題。

[full-repo-codebase-file-invento...](#)

3. Status vocabulary 是否修正？

有，這次通過。

這次 inventory 不再使用上一版的：

```
[x] Read
[-] Binary skipped
[-] Not applicable
```

而是改成：

- Read
- Too large – sampled with explanation
- Listed only – binary/image/model

例如 CSV eval / monitor logs 被標成 **Read**，大型 episode logs 被標成 **Too large – sampled with explanation**，而圖片與模型則標成 **Listed only – binary/image/model**。

[full-repo-codebase-file-invento...](#)

[full-repo-codebase-file-invento...](#)

這符合你要求的狀態集合。

4. CSV / YAML / logs 是否不再被標成 Not applicable？

有，這次通過。

前一版最大問題是 CSV / logs 被標成 **Not applicable**。這次已修正：

- dqn_eval_s1.csv
- dqn_eval_s2.csv
- dqn_train_s1_seed42.monitor.csv
- dqn_train_s2_seed42.monitor.csv

都標成 **Read** / **Full read**。

[full-repo-codebase-file-invento...](#)

大型 episode logs 則標成 **Too large – sampled with explanation**，並在 notes 中寫明約 3000 lines、已檢查 header 與 10 representative rows，且說明這些是 frozen output data，不影響 Step 2。

[full-repo-codebase-file-invento...](#)

這次不再用 **Not applicable** 模糊帶過，通過。

5. 是否真的補到「完整 repo / codebase」的可審計程度？

大致通過。

Inventory 已列出：

- `.agent/skills`
- `.agent/workflows`
- `docs`
- `experiments`
- `figures`
- `reports/handoff`
- `experiments/drl/models`
- OpenSpec Change 01-05
- Phase 3 / Phase 4 source code
- `scripts/phase3`
- `scripts/phase4`
- `proposal / pdr / assets` 等 artifacts

例如 source code 與 scripts 部分已列出：

- `src/ns3/baseline-benchmark.cc`
- `src/ns3/opengym-congestion-env.cc`
- `src/agents/eval_dqn.py`
- `src/agents/train_dqn.py`
- `src/analysis/compare_dqn_baseline.py`
- `src/gym_env/ns3_congestion_env.py`
- `src/gym_env/smoke_test.py`
- `scripts/phase3/analysis.py`
- `scripts/phase3/baseline_runner.sh`
- 多個 `scripts/phase4/*.sh`

而且這些都標成 **Read / Full read**，不是上一版那種代表性檔案式描述。

[full-repo-codebase-file-invento...](#)

[full-repo-codebase-file-invento...](#)

這已經足以證明 Gemini 有做 repo-wide audit，而不是只看兩個 commit 或幾份 README / handoff。

6. Addendum 是否補滿 17 章？

有，這次通過。

`gemini-full-repo-codebase-audit-addendum.md` 已經補出完整結構，從：

1. Executive Summary
2. Why Previous Audit Was Insufficient
3. Repository Inventory Summary
4. Root and Governance Understanding
5. OpenSpec Changes Understanding
6. Phase 3 Baseline Code Understanding
7. Phase 4 OpenGym Environment Code Understanding
8. Phase 4 DQN Training / Evaluation Code Understanding
9. Analysis and Plotting Code Understanding
10. Scripts Understanding
11. Data / Artifact Understanding
12. Result and Limitation Understanding
13. Files I Must Not Modify in Step 2
14. Working Tree Hygiene
15. Commit Summary
16. Readiness for Step 2
17. Audit Checklist

前半段已補上前一版不足、inventory summary、OpenSpec changes、Phase 3 baseline code、Phase 4 OpenGym / DQN 等理解。

[gemini-full-repo-codebase-audit...](#)

[gemini-full-repo-codebase-audit...](#) [gemini-full-repo-codebase-audit...](#)

後半段也有補上 analysis / scripts / data / limitations / Step 2 boundary / hygiene / readiness / checklist。

[gemini-full-repo-codebase-audit...](#)

[gemini-full-repo-codebase-audit...](#) [gemini-full-repo-codebase-audit...](#)

這次已達到 Step 1B 補救要求。

7. Step 2 允許修改範圍是否有鎖住？

有，通過。

Addendum 明確寫出 Step 2 只允許修改六個目標：

```
reports/handoff/change05-validation-note.md
openspec/changes/reporting-figures-and-demo/tasks.md
openspec/changes/reporting-figures-and-demo/specs/result-interpretation.md
openspec/changes/reporting-figures-and-demo/specs/risk-register.md
```

```
openspec/changes/reporting-figures-and-demo/specs/readme-finalization.md
openspec/changes/reporting-figures-and-demo/specs/ppt-package.md
```

並且明確列出 `README.md`、`experiments/`、`figures/`、`src/`、`scripts/`、非 Change 05 的 OpenSpec 文件都不得修改。

[gemini-full-repo-codebase-audit...](#)

Inventory 也對 Change 05 中可在 Step 2 修正的幾個檔案標為 `Allowed to Modify in Step 2? = Yes`，例如 `ppt-package.md`、`readme-finalization.md`、`result-interpretation.md`、`risk-register.md`、`tasks.md`。

[full-repo-codebase-file-invento...](#)

這正好符合下一階段修三個瑕疵的需求。

8. 仍有兩個非阻塞小瑕疵

這次可以通過，但我仍然要誠實標出兩個小問題。

A. Inventory summary 和 addendum summary 的總數不一致

Addendum 裡寫：

```
Total files scanned: 809
```

但 inventory 本體寫：

```
Total files scanned: 808
```

兩者差 1。

[gemini-full-repo-codebase-audit...](#)

[full-repo-codebase-file-invento...](#)

這是 metadata consistency 小瑕疵，不影響 Step 2，因為 Step 2 是修 Change 05 三個瑕疵，不會依賴這個總數做技術決策。

B. Addendum 的 commit summary 還是預提交語氣

Addendum section 15 寫：

```
new remediation commit hash: 即將產生的新 hash
pushed to GitHub: Yes (即將推送)
```

但現在 commit 已經產生，而且你給的 hash 是 `336c02d...`。

[gemini-full-repo-codebase-audit...](#)

這也是非阻塞小瑕疵。理想上它應該回填 commit hash，但不值得再擋一次 Step 2。

9. 最終驗收判定

Step 1B 補救版：驗收通過

項目	判定
commit message 是否符合指定	✓ 通過
是否只修 audit 文件	✓ 通過
inventory 是否九欄	✓ 通過
status vocabulary 是否符合	✓ 通過
CSV / YAML / logs 是否不再 Not applicable	✓ 通過
binary / image / model 是否正確 listed only	✓ 通過
addendum 是否補滿 17 章	✓ 通過
是否有實質技術理解	✓ 通過
是否明確禁止 Step 2 亂改其他檔案	✓ 通過
是否能證明 Gemini 不是只看兩個 commit	✓ 通過
是否完全零瑕疵	⚠ 有 2 個非阻塞 metadata 小瑕疵
是否可正式推進 Step 2	✓ 可以

10. 是否能正式推進到「修 Change 05 三個交接瑕疵」？

可以。

現在可以正式進入：

Step 2：修 Change 05: reporting-figures-and-demo 三個交接瑕疵

Step 2 只能修：

1. 新增 `reports/handoff/change05-validation-note.md`
2. 同步 `openspec/changes/reporting-figures-and-demo/tasks.md`
3. 修正 Change 05 幾個 spec markdown formatting 問題

Step 2 仍然不能：

- 進 Phase 5
- 改 README
- 改 source code
- 改 scripts
- 改 CSV
- 改 figures

- 改 models
 - 重訓 DQN
 - 重跑 baseline
 - 加 PPO / IPFS / QUIC / multi-agent / multi-path
-

最終結論

可以放行 Step 2。

我會正式標記為：

Gemini Full Repository Codebase Audit Addendum：通過。

可以進入「Step 2：修 Change 05 三個瑕疵」。

但 Phase 5：Final Reporting / Demo / PPT Package Implementation 仍然不得開始，必須等 Change 05 修正完美並再次驗收。